A Fine-Scale Segmentation Method for Individual Rubber Trees Based on UAV LiDAR Point Cloud

Li Li, Zilin Ye, Hao Li, Miying Yan, Guoxiong Zhou, Xiangjun Wang, Hengrui Wang, and Mingjie Lv

***Abstract*—As a key tropical economic crop, rubber trees play a vital role in both the global rubber industry and the health of ecological systems. Fine-grained segmentation of rubber tree point clouds is essential for accurately extracting structural parameters and achieving effective monitoring and management. However, existing unsupervised segmentation methods are often affected by ground noise and overlapping tree crowns, leading to suboptimal segmentation results and posing significant challenges for individual rubber tree segmentation. To address these issues, this study proposes a fine-grained segmentation network for rubber trees based on UAV LiDAR point clouds, termed RTreeNet. First, we designed a multiscale feature aggregation (MSFA) module to tackle the issue of leaf overlap by capturing geometric features at the edges of tree crowns. Second, we proposed a cosine-space cross attention (CSCA) module, which calculates the cosine similarity of vertical and horizontal features for each point, effectively eliminating interference from ground noise. Additionally, a adaptive coati particle optimization algorithm (ACPA) was proposed to determine the optimal learning rate for the network, further enhancing segmentation accuracy. The experimental evaluation demonstrates that the proposed RTreeNet outperforms seven state-of-the-art (SOTA) point cloud segmentation architectures and four conventional segmentation algorithms on our custom dataset, achieving a mean intersection over union (mIoU) of 86.3% and an F-score of 92.5%. In the generalization experiment, RTreeNet showed high accuracy and stability on three public datasets. The method also measured the specific structural parameters (tree height, crown diameter, and breast diameter) of rubber trees in the two regions, providing strong technical support for the refined management of rubber trees, agricultural planning, pest control, and rubber yield prediction.**

***Index Terms*—Adaptive coati particle optimization algorithm, cosine-space cross attention (CSCA), multiscale feature aggregation (MSFA) module, rubber tree fine segmentation.**

Received 28 December 2024; revised 23 March 2025, 4 May 2025, 8 June 2025, and 18 July 2025; accepted 24 July 2025. Date of publication 29 July 2025; date of current version 15 August 2025. This work was supported in part by the Hainan Province Science and Technology Special Fund under Grant ZDYF2025XDNY113, in part by the Central Public-Interest Scientific Institution Basal Research Fund under Grant 1630032022007, and in part by the Special Fund for Hainan Excellent Team "Rubber Genetics and Breeding" under Grant 20210203. *(Corresponding authors: Guoxiong Zhou; Xiangjun Wang.)*

Li Li, Zilin Ye, Hao Li, Miying Yan, Guoxiong Zhou, and Hengrui Wang are with the Institute of Artificial Intelligence Application, Central South University of Forestry and Technology, Changsha 410004, China (e-mail: 20231200570@csuft.edu.cn; loyzer@163.com; 20231100414@csuft.edu.cn; 415888273@qq.com; T20060599@csuft.edu.cn; 20231200564@csuft.edu.cn).

Xiangjun Wang is with the Rubber Research Institute, Chinese Academy of Tropical Agricultural Sciences, Haikou 571101, China (e-mail: hnwxj@catas.cn).

Mingjie Lv is with the School of Automation, Central South University, Changsha 410083, China (e-mail: lvmingjie@csu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2025.3593292

I. Introduction

RUBBER trees (Hevea brasiliensis Muell. Arg.) are an important economic crop widely cultivated in tropical regions, primarily known for the natural rubber extracted from their bark. Natural rubber has excellent physical properties and is widely used in tires, footwear, medical devices, and other daily products, making a significant contribution to the global economy. During the growth of rubber trees, accurately identifying and segmenting individual trees is fundamental to extracting key growth parameters such as tree height, trunk diameter, and crown width. These parameters not only aid in scientifically assessing the growth status of rubber trees, but also provide reliable data for agricultural management, pest control, and rubber yield prediction [1]. Therefore, developing efficient individual rubber tree segmentation technology is not only beneficial for optimizing rubber plantation management, but also promotes the development of smart and precision agriculture, enhancing the efficiency and sustainability of rubber production.

With the advancement of computer vision technology, tree segmentation methods based on 2-D images have been widely applied. For instance, the Otsu algorithm is a commonly used adaptive threshold segmentation method for image binarization, which selects the optimal threshold by maximizing interclass variance. Li and Ye [2] achieved multilevel segmentation of rubber tree images through a combination of an improved Otsu method and a snake optimizer. Xie et al. [3] used a dung beetle Otsu (DBO) method to enhance image segmentation, addressing rubber tree diseases through advanced threshold techniques. Although these methods have achieved good segmentation results under certain conditions, they generally rely on predefined thresholds or heuristic rules, making it difficult to adaptively handle the highly diverse images of rubber trees. With the development of deep learning [4], [5], [6], [7], convolutional neural networks (CNNs) have become the mainstream method for image segmentation. For example, Wang et al. [8] identified trunk locations using Faster R-CNN and combined it with a region-growing algorithm (RGA) for canopy segmentation. Chi et al. [9] employed Mask R-CNN for efficient tree instance segmentation. While these deep learning methods can improve segmentation accuracy and extract certain structural parameters of trees (e.g., crown diameter and tree height), 2-D images are susceptible to multiple factors such as image quality, shooting angle, and lighting conditions, which can lead to the loss of spatial structure and depth information of trees. In contrast, 3-D

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

point cloud data can provide more complete structural information.

To overcome the limitations of 2-D tree images, researchers have shifted their focus to processing 3-D point cloud data and have proposed various traditional algorithms [10], [11], [12], [13]. Among the traditional methods, K-means, region growing, multilevel adaptive, and mean shift algorithms demonstrate varying effectiveness across different tree species segmentation studies. Morsdorf et al. [14] applied K-means clustering to segment individual trees in mountain pine forests, extracting tree positions, heights, and crown widths by clustering 3-D coordinates, achieving an R^2 value of 0.92 for tree height estimation. Lee et al. [15] proposed an adaptive clustering algorithm based on region growing to directly process LiDAR point clouds in pine forests, effectively overcoming fixed-window limitations and attaining 95.1% tree detection accuracy. The multilevel adaptive method developed by Hui et al. [16] for coniferous forests integrates the advantages of grid-based and point-based approaches to automatically estimate crown width parameters, yielding an $F1$ score of 0.84 and significantly reducing both omission and commission errors. Chen et al. [17] implemented an enhanced mean shift clustering technique for multilayered coniferous and broad-leaved mixed forests, innovatively incorporating auxiliary trunk detection to improve segmentation accuracy in complex forest environments to 91.1% recall and 93.5% precision, demonstrating robust performance in heterogeneous forest stands. Although traditional algorithms have made certain progress in processing 3-D point cloud data, they often rely on fine parameter tuning and exhibit poor robustness to noise, limiting their performance in complex forest environments. In contrast, deep learning methods can automatically extract features without manual parameter settings, demonstrating higher accuracy and automation in complex scenes. PointNet [18] directly processes unstructured point cloud data using multilayer perceptrons (MLPs) to extract global and local features, avoiding the reliance on feature projection typical of traditional methods while improving computational efficiency. PointNet++ [19] introduced a hierarchical sampling mechanism to further enhance the capture of local geometric information. The Forest-PointNet proposed by Ma et al. [20] utilized spatial and shape features of point clouds to achieve vertical segmentation of ground, shrubs, trunks, and leaves in complex forest scenes, demonstrating strong environmental adaptability. Kim et al. [21] developed a model based on PointNet++ that achieved automatic segmentation of crowns, trunks, and branches. Building on these advancements, researchers have proposed more complex deep learning models to enhance accuracy and efficiency. For example, the two-stage segmentation method proposed by Chang et al. [22] first locates the overall tree area through a semantic segmentation module and then uses an instance segmentation module to finely separate individual trees, addressing the issue of extraction omissions caused by unclear trunk boundaries. However, the modular design has led to decreased efficiency. The TLS2trees method developed by Wilkes et al. [23] combines deep learning with geometric analysis techniques, demonstrating good scalability when processing large-scale point clouds, but its limited reliance on deep learning modules restricts segmentation accuracy in complex scenes. The TreeLearn method proposed by Henrich et al. [24] effectively separates overlapping crown areas by combining semantic and instance segmentation techniques, reducing reliance on manual feature design. However, challenges still exist due to prediction errors causing inaccurate point cloud projections. These errors can lead to initial clustering and instance assignment mistakes, particularly at crown boundaries or low-hanging branches.

The aforementioned research provides valuable insights for the fine segmentation of rubber trees. However, the fine segmentation of rubber tree point clouds still faces three significant challenges.

1) The overlap of crown leaves is a key challenge in the segmentation of rubber tree point clouds. In regions where leaves overlap, it is often difficult to effectively separate the leaves due to weakened gradient information or unclear boundaries, leading to incomplete segmentation or boundary confusion (as shown in Fig. 1(a)).
2) Ground noise is a significant interfering factor in the segmentation of rubber tree point clouds, originating from low shrubs and weeds (as shown in Fig. 1(a)). Due to the similar heights of ground points and the lower parts of tree trunks, combined with uneven terrain and feature similarity, it becomes challenging to distinguish target points from noise points.
3) Determining the most suitable learning rate during the training process of neural networks is complex. Commonly used learning rate optimizers (such as exponential decay and step decay) exhibit limitations in sensitivity to learning rates in complex tasks, resulting in a decline in model performance.

To address the issue of crown leaf overlap, Harikumar et al. [25] redefined the 3-D cloud segments using a crown height model segmentation technique, accurately identifying primary and secondary crowns in the new projection space. However, this method requires high quality and resolution of point cloud data, as low-quality data may lead to inaccurate results. Hao et al. [26] proposed a hierarchical region merging algorithm that generates candidate crown regions through initial region growing and progressively merges overlapping or adjacent crowns to achieve precise segmentation. Although the results are promising, the algorithm may be time-consuming when processing large-scale data and has high computational resource demands. Hu et al. [27] designed a deep learning framework based on conditional generative adversarial networks (cGANs), which, combined with data augmentation techniques, effectively extracts crown areas in complex scenes. However, this method is highly dependent on the quality and diversity of the training data, and its effectiveness may be limited if the training data are insufficient. Therefore, it is necessary to propose a solution that can balance efficiency and accuracy while addressing the issue of overlapping tree crown leaves.

To address the issue of ground noise, Duan et al. [28] proposed a noise removal method that combines grid prin-

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

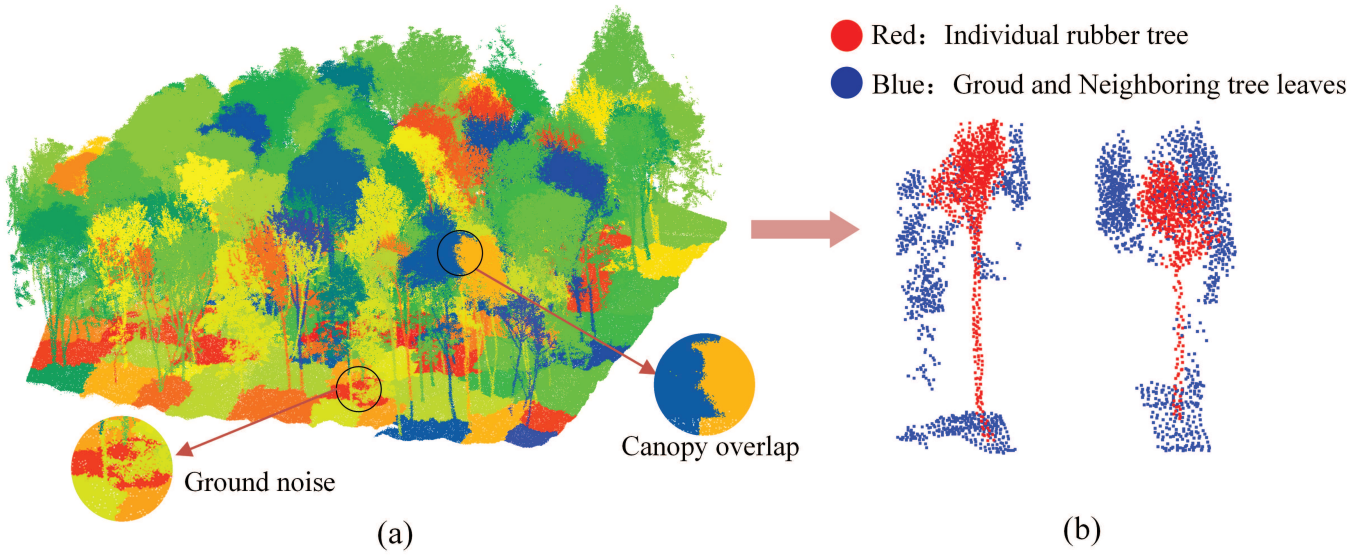


Fig. 1. (a) Two challenges encountered in individual tree segmentation using unsupervised learning methods. (b) Rough segmentation of a single tree point cloud as input data.

cipal component analysis (PCA) and ground stitching. This method performs well in handling complex terrain but has high computational resource demands, which may affect processing efficiency. Diaz et al. [29] designed a voxel-based ground filtering algorithm that divides point clouds into uniform 3-D grids (voxels) and quickly filters ground points by analyzing the lowest points and height distribution of each voxel. Despite its high computational efficiency, this method may require more refined parameter adjustments to ensure effectiveness when processing high-density or complex terrain. Ren et al. [30] proposed the adaptive patchwork algorithm, which utilizes the intensity, height, and positional features of point clouds to remove noise points while efficiently processing large-scale point cloud datasets through multiscale voxelization segmentation. Although this algorithm performs excellently in real-time applications, it may require further optimization in highly unstructured or complex environments. Therefore, it is essential to propose a method that effectively integrates the geometric properties of point clouds with multiscale features to efficiently remove ground noise.

To tackle the challenges of selecting and adjusting learning rates, Ioannou et al. [31] proposed the adaptive layerwise learning rate based on Lipschitz constants (AdaLip) method. This is an optimization approach that dynamically adjusts the learning rate by estimating parameter smoothness and is compatible with existing optimization algorithms [such as stochastic gradient descent (SGD) and adaptive moment estimation (Adam)], thereby accelerating model convergence and improving training accuracy. Kabiri et al. [32] introduced the adaptive moment-based Adam (AMAdam) optimization algorithm, which incorporates an adaptive adjustment strategy based on the curvature of the loss function into the traditional Adam optimizer. Both methods enhance training efficiency and effectiveness by adaptively adjusting the learning rate, but their performance may depend on the specific characteristics of the tasks and datasets. Therefore, it is necessary to propose an efficient method for dynamically adjusting the learning rate to balance convergence speed and generalization ability.

Therefore, the main contributions of this article are as follows.

1) In order to enable the network to accurately segment individual rubber trees and extract structural parameters, we used drones equipped with LiDAR cameras to collect high-quality point cloud data in a real rubber plantation environment. Experts manually and precisely annotated 600 rubber tree samples, covering individual rubber trees with varying heights, canopy widths, and leaf overlap, among other interfering factors.
2) To address the issue of overlapping canopy leaves, we proposed a multiscale feature aggregation (MSFA) that combines farthest point sampling (FPS) and k-nearest neighbor (KNN) algorithms to accurately select representative clusters and their neighboring clusters. This approach captures the geometric information of canopy edges during the encoding process, thereby improving the efficiency and accuracy of feature extraction.
3) To tackle the problem of ground noise, we introduced a novel cosine-space cross-attention (CSCA) method. This method first extracts the semantic context features of the point cloud through the self-attention mechanism, then calculates the cosine similarity between the feature vectors in the horizontal and vertical dimensions, respectively, and performs weighted aggregation on the geometric relationship of neighboring points so as to accurately capture the geometric differences between the ground and the end of the rubber tree trunk and effectively eliminate the interference of ground noise on the segmentation results.
4) To determine the optimal learning rate, we proposed a adaptive coati particle optimization algorithm (ACPA).

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

This algorithm mimics the hunting and evasive behavior of the coati in nature, aiming to efficiently explore initial solutions and pass them to the particle swarm. By utilizing information exchange among particles and updating their positions and velocities, the algorithm accelerates convergence to the global optimal solution, resulting in a 14.2% increase in training speed. It effectively explores and utilizes the optimal learning rate, enhancing the network's convergence speed and adaptability to complex problems.

5) To validate the effectiveness and novelty of the network, a series of experiments was conducted. First, ablation studies and module effectiveness experiments were performed on RTreeNet to assess the capabilities of each module. Second, the model is compared with the most advanced point cloud segmentation networks and classic traditional segmentation algorithms to demonstrate its state-of-the-art (SOTA) performance. In addition, experiments are conducted on three public datasets to evaluate the generalization ability of the model. Based on good segmentation results, we measured the structural parameters of rubber trees, including height, diameter at breast height (DBH), and crown diameter, achieving an R^2 of 99% for tree height. This method is suitable for forest resource inventory and health monitoring, providing an effective means to deepen our understanding of forest structure. To facilitate reproducibility and further research development, the complete implementation code for RTreeNet is made publicly available at https://github.com/Chocolate-37/RTreeNet.

II. Materials

A. Study Area

The study area is located in the northwest of Danzhou, Hainan Province (latitude 19°31′50.59″ N and longitude 109°28′52.62″ E), within the experimental region of the Rubber Research Institute of Chinese Academy of Tropical Agricultural Sciences, as shown in Fig. 2(a). Danzhou is situated in the East Asian subtropical region, with an average annual temperature ranging from 22.5 °C to 25.6 °C. The climate is warm and stable, with a small temperature difference throughout the year; January is the coldest month, while July is the hottest. The annual sunshine duration is between 1780 and 2600 h, and the annual precipitation ranges from 900 to 2400 mm, providing ample moisture. These conditions create an ideal environment for the growth of rubber trees in terms of light, moisture, and temperature.

B. Data Acquisition

During the data collection, we used a DJI M300 RTK drone from Shenzhen DJI Innovation Technology Company Ltd., as shown in Fig. 2(d), which had a CBI Lite LiDAR measurement camera from Chengdu AuroTech Company Ltd., with specific details listed in Table I. The acquisition of rubber forest point cloud data for this study was technically supported by the Rubber Research Institute of Chinese Academy of Tropical Agricultural Sciences and was carried out on March 5, 2023.

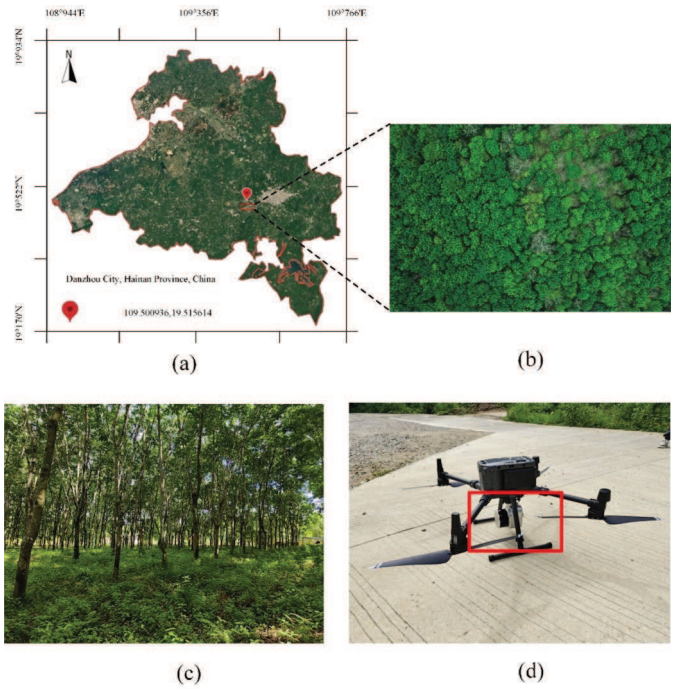


Fig. 2. (a) Location of the study area. (b) and (c) Actual photographs of the rubber forest. (d) Data collection equipment.

TABLE I
Specific Parameters of CBI-Lite LiDAR Measurement Camera

Camera Parameters	Value
Scanning Range	120m (@Reflectivity = 60%)
Scanning Field of View	360°
Weight	1.28kg
Measurement Range	0-120m
Measurement Accuracy	5mm (within 30m range)
Repeatability Accuracy	10mm
Scanning Frequency	5Hz-20Hz
Laser Wavelength	905nm
Echo Number	Dual Echo
Maximum Effective Rate	640,000 points/second
Camera	20 million pixels
Field of View	84°

The sampling locations are shown in Fig. 2(b) and (c). The UAV flew at an altitude of 80 m, maintaining a distance of 50–60 m from the canopy, with an average flight speed of 6 m/s, a flight line spacing of 10 m, and a lateral overlap rate of approximately 65%. High-resolution 3-D point cloud data of Hainan rubber plantation was acquired through LiDAR, clearly capturing the rubber tree canopies, vertical structures, and lower features. The collected data had a point density of approximately 10 667 points/m^2; the experimental forest consisted of even-aged trees, with tree heights ranging from 17.28 to 26.02 m, and a mean tree height of 21.65 m. Table II presents the main statistical characteristics of the dataset.

C. Data Preprocessing

In order to obtain high-quality rubber tree point cloud data, this study adopted a systematic preprocessing process: first, the original point cloud was subjected to a statistical outlier filtering algorithm [33] to remove noise points; then, the cloth

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

TABLE II
STATISTICAL CHARACTERISTICS OF THE EXPERIMENTAL DATASET

Parameters	Value
Point cloud density	10,667 points/m²
Tree height range	17.28-26.02m
Mean tree height	21.65m
Tree age	15 years
Data type (sensor)	UAV-LiDAR (CBI Lite)

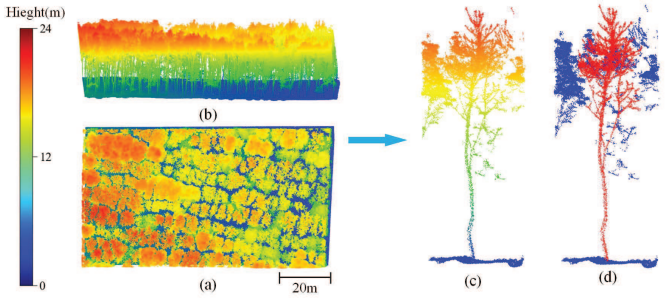


Fig. 3. Data preprocessing flowchart. (a) Top view of the rubber forest point cloud in the study area. (b) Side view of the rubber forest point cloud in the study area. (c) Example of a single rubber tree manually segmented and extracted from the original point cloud. (d) Result after the segmented rubber tree is labeled, where red represents the target rubber tree point cloud and blue represents the interference point cloud.

simulation filter (CSF) algorithm [34] was used to separate ground and nonground points; a digital elevation model (DEM) was generated based on the extracted ground points; the elevation difference between the point cloud elevation and the DEM was calculated to achieve elevation normalization; and finally, considering the unique stand structure characteristics of the rubber forest, a method combining height threshold and point density analysis [35] was used to filter out the understory vegetation point cloud, and relatively pure rubber tree point cloud data was obtained. Fig. 3(a) and (b) shows the top view and front view of part of the rubber forest, respectively, clearly showing the spatial distribution pattern and vertical height gradient of the rubber tree.

Single tree segmentation and data annotation were completed by experts [36] at Hainan Rubber Tree Research Institute using CloudCompare software [37]. The specific process is as follows: observe the point cloud from the top view shown in Fig. 3(a), perform preliminary regional division based on the crown morphological characteristics and tree gaps, and obtain coarse segmentation data containing the target tree, as shown in Fig. 3(c). Then, the preliminary divided areas were fine-tuned from multiple perspectives (including top view, front view, and side view), with special attention paid to the boundary determination of the crown overlapping area, and accurate segmentation was performed based on the density change of the point cloud and the geometric continuity characteristics. Finally, the segmentation results were binary labeled, with the target rubber tree points labeled as 1 (displayed in red) and the background points (including the ground, adjacent tree parts, and weeds) labeled as 0 (displayed in blue), as shown in Fig. 3(d). All the labeling results were reviewed and verified by three experienced rubber tree research experts to ensure that the labeling accuracy reached more than 95%.

After the above preprocessing process, 600 sample trees representing different heights, crown widths, and background complexities were selected from thousands of segmented rubber trees to construct a dataset, which was divided into training and test sets in an 8:2 ratio, and a fivefold cross-validation method was used to improve the robustness of the model.

III. METHODS

This article presents a novel network, RTreeNet, designed for the fine segmentation of individual rubber trees and the extraction of their structural parameters, as illustrated in Fig. 4(a). The network comprises several modules, including Point Embedding, CSCA, MSFA, Decoding, Mapping, and Classifier. First, the Point Embedding module transforms the geometric information of the point cloud into high-dimensional features, laying the groundwork for subsequent processing. Next, the CSCA module (as shown in Fig. 4(b)) employs a cross-scale attention mechanism to integrate global and local features, effectively separating ground noise. Following this, the MSFA module (illustrated in Fig. 4(c)) utilizes multiscale feature fusion to efficiently segment overlapping canopies. Finally, the Decoding and Mapping modules progressively restore the structural features of the point cloud, while the Classifier module maps these features to the target space, achieving precise segmentation. The organic integration of these modules facilitates the efficient processing and accurate segmentation of point cloud data.

A. MSFA

To address the challenge of overlapping rubber tree leaves, we propose a MSFA module, as shown in Fig. 4(c). The basis of most point cloud processing networks [18] is a series of core algorithm units, including FPS, KNN, and pooling operations (extracting salient features). We represent the input point cloud as $\{q_i, r_i\}_{i=1}^{M}$, where M represents the number of points in the original point cloud, $q_i \in \mathbb{R}^{1\times 3}$ represents the coordinates of point i, and $r_i \in \mathbb{R}^{1\times C}$ is the feature of point i. The FPS downsampling formula is as follows:

$$\{q_c, r_c\}_{c=1}^{\frac{M}{2}} = \mathrm{FPS}\left(\{q_i, r_i\}_{i=1}^{M}\right) \tag{1}$$

where q_c represents the center point coordinate set selected by FPS, and r_c represents the feature vector corresponding to q_c. The KNN algorithm identifies k spatial neighbors for each center point c from the original point set M to form a local 3-D region. The calculation process formula is as follows:

$$\mathcal{L}_c = \mathrm{kNN}\left(q_c, \{q_i\}_{i=1}^{M}\right) \tag{2}$$

where $\mathcal{L}_c \in \mathbb{R}^{k\times 1}$ represents the index of the KNN points. The geometric pattern of each local neighborhood $\mathcal{L}_c$ is extracted by a sophisticated learnable operator $\Phi(\cdot)$, where $\Phi(\cdot)$ is a neural network layer containing learnable parameters, and finally, the most significant features are extracted from the features of different points by the maximum pooling MaxP($\{\cdot\}$). The formula is as follows:

$$r_c^a = \mathrm{MaxP}\left(\Phi\left(r_c, r_j\right) \mid j \in \mathcal{L}_c\right). \tag{3}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

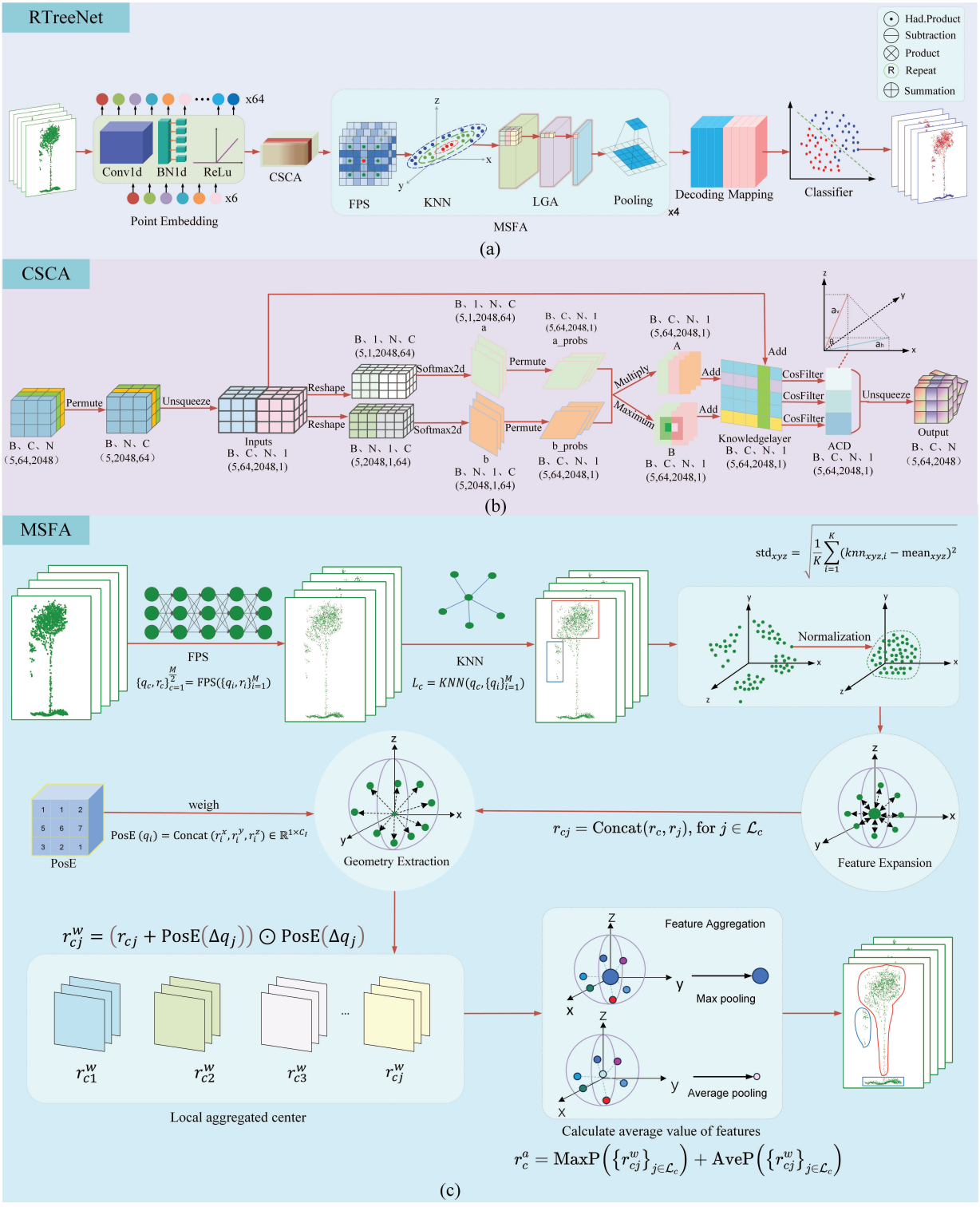


Fig. 4. (a) Overall architecture of the RTreeNet network, including core modules such as point embedding, feature extraction, decoding, and classification, shows the complete processing flow from raw point cloud to precise segmentation. (b) CSCA module structure, which realizes the effective separation of ground noise through the CSCA mechanism. (c) MSFA module structure, which adopts a multiscale feature aggregation strategy to solve the effective segmentation of the overlapping area of the crown.

Among them, r_c^a represents the center point feature after aggregation through the maximum pooling operation. However, we are thinking about the nonparametric representation of the learnable operator $\Phi(\cdot)$. This study uses the 3-D position encoding function PosE$(\cdot)$ based on trigonometric functions to replace $\Phi(\cdot)$. This design has two advantages: first, by eliminating the need for parameter updates and gradient calculations, the computational complexity is significantly reduced; second,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

trigonometric functions can accurately encode the absolute and relative position information of the point cloud, which is especially suitable for geometric feature extraction in the boundary area of rubber trees. The specific formula is as follows:

$$\mathrm{PosE}\left(q_i\right) = \mathrm{Concat}\left(r_i^x, r_i^y, r_i^z\right) \in \mathbb{R}^{1\times C_I}. \tag{4}$$

For the original point i, its coordinates are $q_i = (x_i, y_i, z_i) \in \mathbb{R}^{1\times 3}$, and we use sine and cosine trigonometric functions to embed it into a C_I-dimensional vector. On this basis, we introduce PosE(·) to explore local patterns to achieve the following three functions.

1) *Function Expansion:* As the network grows, the features of each point need to obtain a larger receptive field, and the feature dimension needs to be increased to capture 3-D semantic information. For each center point $\{q_c, r_c\}$ and its neighborhood $\{q_c, r_j\}_{j\in\mathcal{L}_c}$, we implement this feature expansion by simply concatenating the neighborhood feature r_j with the center feature r_c along the feature dimension, where r_{cj} represents the expanded feature of each neighboring point. The calculation process formula is as follows:

$$r_{cj} = \mathrm{Concat}\left(r_c, r_j\right) \ , \text{ for } \ j \in \mathcal{L}_c \tag{5}$$

2) *Geometric Feature Extraction:* To illustrate the spatial distribution of k neighbors in the local region, we assign weights to each r_{cj} using relative position encoding. Specifically, their coordinates are normalized by applying the mean and standard deviation, denoted as $\{\Delta q_j\}_{j\in L_c}$. Subsequently, the feature weights of the neighboring points k are calculated as follows:

$$r_{cj}^w = \left(r_{cj} + \mathrm{PosE}\left(\Delta q_j\right)\right) \odot \mathrm{PosE}\left(\Delta q_j\right) \tag{6}$$

where $\odot$ denotes the elementwise multiplication. Due to feature expansion, the feature dimensions of q_{cj} are set to four stages that match q_{cj}, specifically $2C_I$, $4C_I$, $8C_I$, and $16C_I$. In this way, the local geometric shape of the region, i.e., the relative positional information of neighboring points PosE(Δq_j), can be implicitly encoded into the features without the need for any learnable parameters.

3) *Feature Aggregation:* We utilize max pooling and average pooling for local feature aggregation. The computation is expressed as follows:

$$r_c^a = \mathrm{MaxP}\left(\left\{r_{cj}^w\right\}_{j\in\mathcal{L}_c}\right) + \mathrm{AveP}\left(\left\{r_{cj}^w\right\}_{j\in\mathcal{L}_c}\right) \tag{7}$$

where MaxP({·}) and AveP({·}) represent the permutation-invariant operations that aggregate features of adjacent points from multiple perspectives; specifically, these operations remain unaffected by the order of the input points. Here, we obtain a new local aggregate center feature r_c^a, which contains important geometric information of the local area around the point. It will be input to the next stage of processing.

In summary, MSFA uses FPS to select local center points from the original point set. Then, KNN is used to group the neighboring points of each center point to form a local 3-D region. The geometric features of each region are extracted through the position encoding function, and the features are aggregated through the pooling technique. These building blocks do not rely on parameter updates during training but are implemented based on predefined mathematical rules. The multilayer network architecture is used to further aggregate these features hierarchically, expand and weight the features of adjacent points, expand the receptive field, and capture the geometric information of the edge contour of the rubber tree to solve the leaf overlap problem. The effectiveness of the MSFA module is demonstrated in Section IV-C.1.

B. CSCA

In the rubber tree segmentation task, ground noise (such as low shrubs and weeds) is often adjacent to the trunk or canopy, making it difficult to distinguish target points from noise points, thereby increasing the difficulty of segmentation. To address this issue, we propose the CSCA module. First, the input features X undergo three linear transformations to generate the query matrix Q, key matrix K, and value matrix V. Here, W_Q, W_K, and W_V are the learnable weights used to map the input features into different feature spaces, with an output dimension of d_k. The implementation process is as follows:

$$Q = XW_Q, K = XW_K, V = XW_V. \tag{8}$$

Then, the attention scores are obtained by calculating the dot product between the query matrix and the key matrix, and this is expressed as follows. These scores are subsequently scaled to prevent gradient vanishing or exploding issues

$$\mathrm{Attention}\left(Q, K, V\right) = \mathrm{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{9}$$

In addition, this module incorporates position encoding directly extracted from point coordinates to efficiently modulate the attention mechanism, as shown in the following:

$$\text{Positional Encoding}\left(p_i, p_j\right) = \mathrm{MLP}\left(p_i - p_j\right). \tag{10}$$

The relative position vector $(p_i - p_j)$ between points p_i and p_j is computed and then fed into an MLP. This relative position vector passes through a series of fully connected layers and nonlinear activation functions to produce a new feature vector, which serves as the output of the positional encoding. By introducing the relative positional information between points, the positional encoding enhances the model's ability to effectively capture the spatial structure of the point cloud.

Based on this, the cross-attention mechanism further combines and processes the position encoding with other input features to capture the interrelationships between features. Cross-attention calculates weights in both vertical and horizontal directions, tightly integrating the spatial relationships of the point cloud with feature interactions. This allows the structural graph convolution to output feature maps, represented as $g_{i,j}^h \in R^{H\times W}$, where HW denotes the dimensions of the feature map, and $g_{i,j}^k$ is the input to the horizontal attention unit. The

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

direction and formula for calculating the horizontal weights can be defined as follows:

$$X_h = \frac{e^{(W_h g_{i,j} + b_h)}}{\sum_{i,j} e^{(W_h g_{i,j} + b_h)}} \tag{11}$$

where W_h and b_h are the weight parameters and X_h is the coefficient of attention along the horizontal direction. The weight in the vertical direction is calculated by inversing the characteristic matrix, and the process is represented as follows:

$$g^v_{j,i} = \left(g^v_{i,j}\right)^T \tag{12}$$

where $g^v_{j,i}$ represents the transposed characteristic graph. Therefore, we can obtain the attention weight in the vertical direction in the same way. The formula is as follows:

$$X_v = \frac{e^{(W_v g_{j,i} + b_v)}}{\sum_{j,i} e^{(W_v g_{j,i} + b_v)}} \tag{13}$$

where W_v and b_v are the weight parameters, and X_v is the coefficient along the vertical direction

$$\cos\theta = \frac{\sum_{i=1}^{n} X_i^{(h)} X_i^{(v)}}{\sqrt{\sum_{i=1}^{n} \left(X_i^{(h)}\right)^2} \sqrt{\sum_{i=1}^{n} \left(X_i^{(v)}\right)^2}} \tag{14}$$

$$\begin{cases} T_{X_h}(\zeta) = \begin{cases} 0, & \cos_h \le \text{thr} \\ 1, & \cos_h > \text{thr} \end{cases} \\ T_{X_v}(\zeta) = \begin{cases} 0, & \cos_v \le \text{thr} \\ 1, & \cos_v > \text{thr}. \end{cases} \end{cases} \tag{15}$$

In general, the CSCA module removes the noise or redundant features of rubber trees near the ground by calculating the feature cosine similarity in the vertical and horizontal directions in the point cloud. The core idea is to use the attention mechanism to distinguish the correlation of different spatial directions by weighting the characteristic matrix. Specifically, first calculate the cosine similarity between the query matrix and the key matrix (as shown in (14)). When the cosine similarity is close to 0, it means that the correlation between features is low, which usually means that they are independent in space, and may be ground noise. By setting the threshold value thr (as shown in (15)), we can filter out the features with low correlation and set it to 0, so as to suppress the ground noise. At the same time, features with high similarity are retained. The effectiveness experiment of CSCA is presented in Section IV-C.2.

C. ACPA

Determining the optimal learning rate is crucial for accelerating model convergence, improving accuracy, and enhancing generalization capability. Traditional methods often rely on manual parameter tuning and multiple iterations of training to identify the best value, which is not only time-consuming and labor-intensive but also prone to inaccuracies. To address this challenge, we propose the ACPA, which autonomously explores the optimal learning rate while reducing the likelihood of errors caused by manual parameter adjustments. This algorithm mimics the hunting and evasion behaviors of the long-nosed raccoon in nature to find multiple good initial solutions, which are then passed to particles. By utilizing information exchange among particles, as well as position and velocity update mechanisms, ACPA accelerates the convergence toward the global optimal solution. The pseudocode for the algorithm is shown in Table III, where dim is the search space dimension, lb is the lower bound, ub is the upper bound, objf is the objective function, SearchAgents_no is the number of search agents, Max_pos is the maximum iterations, PSO_iters are the PSO iterations, and the algorithm outputs the global best position and global best fitness. Each long-nosed raccoon acts as a member of the algorithm's population, and its position in the search space determines the value of the decision variables. In the initial phase of ACPA implementation, the positions of the raccoons in the search space are randomly initialized according to the following equation:

TABLE III
PSEUDOCODE FOR ACPA

```
Algorithm: ACPA
Input: dim, lb, ub, objf, SearchAgents_no, Max_pos, PSO_iters
Output: global_best_position, global_best_fitness
Begin
  Initialize X, v, p_best, p_bestFit, g_best, g_bestFit
  Function Evaluate:
    for i from 0 to SearchAgents_no-1:
      Compute fitness
      Update best_positions[i] and best_fitness[i]
      If fitness[i] ¡ global_best_fitness:
      Update global_best_position
      and global_best_fitness
  Function Optimize:
    for t from 0 to Max_pos-1:
      Evaluate()
      for i from 0 to SearchAgents_no-1:
        COA Phase:
          Update positions using COA logic
          Clip positions within bounds
        PSO Phase:
          Update velocities using PSO equations
          Update positions using velocities
          Clip positions within bounds
      Update global best if necessary
  Return global_best_position, global_best_fitness
End
```

$$X_i : x_{i,j} = lb_j + r \cdot \left(ub_j - lb_j\right), i \in [1, N],\ j \in [1, m] \tag{16}$$

where $x_{i,j}$ is the value of the ith long-nosed raccoon on the jth decision variable, N is the number of long-nosed raccoons, m is the number of decision variables, r is the random real number in the interval [0,1], and lb_j and ub_j are the lower and upper limits of the nth decision variable, respectively. In ACPA, the mathematical expression of raccoon population is the population matrix, given as follows:

$$X = \begin{bmatrix} x_{1,1} & \cdots & x_{1,m} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,m} \end{bmatrix}. \tag{17}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

The positions of candidate solutions can lead to different values of the objective function. This relationship is represented as follows:

$$F = \begin{bmatrix} F(X_1) \\ \vdots \\ F(X_i) \\ \vdots \\ F(X_N) \end{bmatrix}. \tag{18}$$

In this context, F represents the objective function vector, and $F(X_i)$ denotes the objective function value calculated based on the position of the coati. In the ACPA algorithm, the quality of candidate solutions is measured by their objective function values; thus, the population members that yield the optimal objective function values are referred to as the best members in the population. The positions of the candidate solutions are updated by simulating the natural behaviors of two types of iguanas, specifically the strategies for predation and the strategies for avoiding predators. Consequently, the population update process is divided into two stages.

In the exploration phase (the strategy of predation and attacking iguanas), the first stage updates the positions of the population by simulating the strategies of coati attacking iguanas. Some coati climb trees to scare the iguanas, while others wait under the trees to hunt the iguanas after they fall. This strategy allows the coati to move to various positions in the search space, thereby demonstrating the global search capability of the ACPA algorithm. In the algorithm design, it is assumed that the position of the best member in the population corresponds to the position of the iguana, with half of the coati climbing the tree and the other half waiting under the tree for the iguanas to fall. The position update formula for the climbing coati is expressed as

$$X_i^{P1} : x_{i,j}^{P1} = x_{i,j} + r \cdot \left(\text{Iguana}_j - I \cdot x_{i,j}\right). \tag{19}$$

When the iguana falls to the ground, its position is randomly placed in the search space. Based on this random position, the raccoons on the ground move in the search space are calculated as

$$\text{Iguana}_j^G = lb_j + r \cdot \left(ub_j - lb_j\right) \tag{20}$$

$$x_{i,j}^{P1} = \begin{cases} x_{i,j} + r \cdot \left(\text{Iguana}_j^G - I \cdot x_{i,j}\right), & \text{if } F_{\text{Iguana}^G} < F_i \\ x_{i,j} + r \cdot \left(x_{i,j} - \text{Iguana}_j^G\right), & \text{else} \end{cases} \tag{21}$$

for $i \in \left[\frac{N}{2} + 1, N\right]$ and $j \in [1, m]$. If the new position improves the objective function value, it is accepted for the update process; otherwise, the raccoon remains in its original position. The calculation is defined as

$$X_i = \begin{cases} X_i^{P1}, & \text{if } F_i^{P1} < F_i \\ X_i, & \text{else.} \end{cases} \tag{22}$$

In the development phase (the process of evading predators), the second stage updates the positions of the coati by simulating their natural behavior when encountering a predator. When a predator attacks a coati, it escapes from its current position to a safe location, reflecting the local search capability of the ACPA. This behavior is simulated by randomly generating a new position in the vicinity of the coati's current location, calculated as follows:

$$\text{lb}_j^{\text{local}} = \frac{\text{lb}_j}{t}, \quad \text{ub}_j^{\text{local}} = \frac{\text{ub}_j}{t}, t \in [1, T]. \tag{23}$$

As shown in the following equation, if the new position improves the objective function value, the new position is accepted for the update:

$$X_i = \begin{cases} X_i^{P2}, & \text{if } F_i^{P2} < F_i \\ X_i, & \text{else.} \end{cases} \tag{24}$$

In particle swarm optimization (PSO), receiving initial solutions from the first two stages is a crucial step. Specifically, the core of the algorithm is to explore the solution space through the cooperation and competition among particles, ultimately finding the global optimal solution. Each particle has its own position and velocity, which are updated using (25) and (26). These particles move within the solution space, continuously updating their positions to approach both their individual historical best positions and the global best position

$$v_i = w \cdot v_i + c_1 \cdot r_1 \cdot \left(p_i^{\text{best}} - x_i\right) + c_2 \cdot r_2 \cdot \left(g^{\text{best}} - x_i\right) \tag{25}$$

$$x_i = x_i + v_i \tag{26}$$

where v_i is the velocity of particle i, x_i is the position of particle i, w is the inertia weight, c_1 and c_2 are the acceleration constants, and r_1 and r_2 are the random numbers within the interval [0, 1]. p_i^{best} represents the historical best position of particle i, while g^{best} denotes the global best position of the population.

Overall, the ACPA algorithm effectively addresses the inefficiencies and manual tuning issues associated with traditional learning rate adjustment methods by simulating the hunting and evasion behaviors of the long-nosed raccoon in conjunction with PSO. This algorithm autonomously explores the optimal learning rate, significantly enhancing the convergence speed and generalization capability of the model. The effectiveness experiment of ACPA is presented in Section IV-C.3.

IV. Experiments

This section validates the effectiveness of RTreeNet through a series of experiments. First, ablation studies and effectiveness experiments are conducted on a self-constructed dataset to assess the contributions of the MSFA, CSCA modules, and ACPA. Next, RTreeNet is trained and validated on the self-constructed dataset, demonstrating its performance advantages over SOTA point cloud segmentation networks. Subsequently, the model is tested on three public datasets to compare its generalization capability. Finally, structural parameters of rubber trees are extracted from the segmentation results, further showcasing the practical value of the network.

A. Experimental Environment and Parameter Setting

The experimental environment and parameter settings are detailed in Table IV, and all subsequent experiments are conducted within this environment. To accelerate convergence

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
EXPERIMENTAL ENVIRONMENT AND PARAMETER SETTING

Experimental Parameters	Setting
Hardware environment	Intel (R) Xeon (R) Gold 6330 CPU NVIDIA RTX 3090 GPU
Software environment	Ubuntu 18.04.6 LTS Python3.8 PyTorch 1.8.1
Optimizer	Adam
Scheduler	ACPA
lb	1.00E-08
ub	5.00E-03
dim	1
Batch Size	5
SearchAgents no	60
num-loops	2
Epochs	540

and enhance model stability, this study proposes an ACPA optimization algorithm as a scheduler for adaptively updating the global learning rate. The search range for the learning rate is constrained between $1e^{-8}$ and $5e^{-3}$ to ensure that the learning rate is neither too low, which would lead to slow training, nor too high, which could cause instability or divergence. The optimization objective function involves only the learning rate as an independent variable, resulting in a knowledge space dimension (dim) of 1. We set the number of search agents (SearchAgents_no) to 60, indicating that the algorithm initializes 60 candidate learning rates in the search space and iteratively determines the optimal learning rate. If the population is too small (<10), the search range is insufficient, leading to a tendency to get stuck in local optima; conversely, if it is too large (>100), the computational overhead increases significantly. The number of iterations for the optimization algorithm (num-loops) is set to 2 to allow for a more comprehensive exploration of the optimization space and to seek better solutions. The training frequency is calculated as (SearchAgents_no * 4 + SearchAgents_no / 2) * num-loops = 540 iterations, with a batch size of 5.

B. Evaluation Metrics

We use the following five evaluation indexes to ensure the accuracy and fairness of the experiment.

Mean intersection over union (mIoU) measures the average overlap between the predicted segmentation results and the ground-truth annotations across all samples. It includes intersection over union (IoU), which calculates the ratio of the intersection area to the union area for each sample between the predicted segmentation result A_{pred} and the ground-truth segmentation result A_{true}. In the rubber tree point cloud segmentation task, a higher mIoU indicates that the model can more accurately distinguish individual rubber trees from the background vegetation and effectively handle complex situations such as crown overlap and ground noise, which helps in evaluating the overall segmentation quality. The calculation formula is represented as

$$\text{IoU} = \frac{A_{\text{pred}} \cap A_{\text{true}}}{A_{\text{pred}} \cup A_{\text{true}}} \tag{27}$$

$$\text{mIoU} = \frac{1}{N}\sum_{i=1}^{N} IoU_i \tag{28}$$

Mean accuracy (mAcc) represents the classification accuracy of the model's predictions, defined as the proportion of correctly classified samples among all samples. This includes true positives (TP), where positive classes are predicted as positive, and true negatives (TN), where negative classes are predicted as negative. Additionally, misclassifications include false positives (FP), where negative classes are predicted as positive, and false negatives (FN), where positive classes are predicted as negative. In point cloud segmentation tasks, a higher mAcc indicates better classification performance across different categories, which helps in comprehensively evaluating the overall performance of the model. The calculation formula is calculated as follows:

$$\text{mAcc} = \frac{1}{N}\sum_{i=1}^{N} \frac{\text{TP}_i + \text{TN}_i}{\text{TP}_i + \text{TN}_i + \text{FP}_i + \text{FN}_i}. \tag{29}$$

Mean precision (mPrec) measures the proportion of actual positive samples among those predicted as positive. It includes TPs, where positive classes are correctly predicted, and FPs, where negative classes are incorrectly predicted as positive. In point cloud segmentation tasks, a higher mPrec indicates that the model can more accurately label target points, which aids in assessing the predictive precision of the model. The calculation formula is expressed as

$$\text{mPrec} = \frac{1}{N}\sum_{i=1}^{N} \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i}. \tag{30}$$

Mean recall (mRec) measures the proportion of actual positive samples that are correctly predicted as positive. This includes TP, where positive classes are correctly predicted, and FN, where positive classes are incorrectly predicted as negative. In point cloud segmentation tasks, a higher mRec indicates that the model is better at comprehensively identifying target objects, which helps in evaluating its detection and coverage capability. The calculation formula is described by

$$\text{mRec} = \frac{1}{N}\sum_{i=1}^{N} \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i}. \tag{31}$$

Mean F-Score (mF-Score) is the weighted harmonic mean of precision and recall, used to provide a comprehensive measure of a model's accuracy and coverage. In point cloud segmentation tasks, the mF-Score helps to balance the tradeoff between precision and recall, making it suitable for evaluations that emphasize both classification accuracy and the comprehensiveness of target detection. The calculation formula is shown as follows:

$$\text{mF-Score} = \frac{1}{N}\sum_{i=1}^{N} \frac{2 \cdot \text{Pre}_i \cdot \text{Rec}_i}{\text{Pre}_i + \text{Rec}_i}. \tag{32}$$

C. Module Effectiveness Experiment

This section provides an in-depth analysis of the evaluation metrics and parameters for the MSFA, CSCA, and ACPA modules within the RTreeNet model. Under the same experimental settings, the effectiveness of these innovative modules has been thoroughly validated. Through a detailed interpretation of the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

TABLE V
RESULTS OF MSFA EFFECTIVENESS COMPARISON

Method	mIoU(%)	mAcc(%)	mPrec(%)	mRec(%)	mF-Score(%)
MLP_blocks	82.5	89.3	90.2	89.3	89.3
EncNP	80.0	89.0	89.0	89.0	89.0
MSFA	**86.3**	**92.5**	**92.7**	**92.5**	**92.5**

TABLE VI
RESULTS OF CSCA EFFECTIVENESS COMPARISON

Method	mIoU(%)	mAcc(%)	mPrec(%)	mRec(%)	mF-Score(%)
Thr0.5_front	**86.37**	**92.53**	**92.71**	**92.53**	**92.53**
Thr0.6_front	85.00	91.48	91.80	91.48	91.48
Thr0.7_front	85.60	91.58	92.03	91.58	91.58
Thr0.5_mid	84.57	90.93	91.48	90.93	90.91
Thr0.5_end	84.15	90.69	91.57	90.69	90.65
CCTA_0.5_front	84.6	91.2	91.5	91.2	91.2

experimental results, we further confirm the key contributions and practical effects of each module in the fine segmentation task of rubber trees.

1) Effectiveness Experiment of MSFA: To evaluate the performance of the MSFA module, we compared it with the encoder modules of different networks, such as MLP_blocks [38] and EncNP [39]. Specifically, we integrated MLP_blocks, EncNP, and MSFA into the RTreeNet architecture and conducted training for each configuration. The experimental results are shown in Table V. The MLP_blocks module offers advantages such as a simple structure, high computational efficiency, and ease of implementation and training. However, it fails to explicitly consider the spatial relationships between points, making it challenging to capture complex geometric structures. Consequently, it struggles to address the significant overlap issues encountered in rubber tree segmentation, presenting certain limitations compared to MSFA. On the other hand, the EncNP module employs a multilevel hierarchical structure and geometric feature extraction. By combining local feature aggregation with global feature aggregation, it can extract feature information from rubber trees to some extent. However, the complex calculations and aggregation operations involved in dealing with geometric relationships significantly increase the computational resource consumption during training and inference. In contrast, the experimental results indicate that the more lightweight MSFA module demonstrates superior performance when integrated into the RTreeNet network.

2) Effectiveness Experiment of CSCA: To verify the effectiveness of the CSCA module, we designed three experiments, which, respectively, explored the setting of the Thr parameters, the position of the CSCA module in the network, and the comparison of different attention mechanisms. As shown in Table VI, the experimental results provide a comprehensive basis for the performance evaluation of the module. In the table, different Thr values (0.5, 0.6, and 0.7) represent the threshold parameter experiments, the position indicators (front, mid, and end) correspond to the module placement experiments, and continuous context threshold-aware (CCTA) [40] represents the comparison with alternative attention mechanisms.

In the CSCA module, the Thr parameter is used to control the sparsity of the feature matrix and the filtering accuracy. By calculating the cosine similarity of the point cloud, feature points that exceed this threshold are retained, while those below it are discarded. This filtering mechanism effectively enhances the model's focus on key features while reducing the interference of irrelevant information on the segmentation results. The experiments indicate that when Thr is set to 0.5, the model performs optimally across all evaluation metrics.

Further research indicates that the position of the CSCA module within the network also has a significant impact on its performance. We conducted experiments by placing the module at the front, middle, and end positions, respectively, with results shown in Table VI. The experimental results demonstrate that placing the module at the front end can better capture the spatial structure of point clouds, particularly excelling in parsing geometric features of near-ground areas. This helps the model accurately identify key information and reduces the impact of ground noise on segmentation results.

In contrast, the CCTA module [40] employs simple convolution and pooling operations and also introduces a threshold filtering mechanism to enhance features. While the CCTA module performs exceptionally well in hyperspectral image classification, the experimental results indicate that its effectiveness in handling ground noise in point clouds is inferior to that of the CSCA module. The CSCA module leverages the powerful feature extraction capabilities of point transformation layers, allowing it to not only clearly capture spatial features within the point cloud but also effectively eliminate irrelevant information. This is particularly evident in its superior performance in removing ground noise.

TABLE VII
RESULTS OF ACPA EFFECTIVENESS COMPARISON

Scheduler	mIoU(%)	mAcc(%)	mPrec(%)	mRec(%)	mF-Score(%)
COA	83.90	90.63	91.15	90.63	90.61
StepLR	83.10	89.94	90.76	89.94	89.93
PSO	71.59	83.07	84.19	83.07	83.06
ACPA	**86.37**	**92.53**	**92.71**	**92.53**	**92.53**

3) Effectiveness Experiment of ACPA: To validate the performance of the ACPA algorithm, we conducted comparative experimental analyses, with the results presented in Table VII. The experiments uniformly employed the Adam optimizer [41], which primarily achieves adaptive optimization by dynamically adjusting the learning rates of the network model's weight parameters and bias parameters. Additionally, we incorporated various learning rate schedulers to dynamically adjust the global learning rate lr further enhancing the model training process.

Among them, the StepLR scheduler reduces the learning rate in proportion with a fixed step size, which is simple and easy to use, but lacks the ability to adapt to the training state in real time. The coatis optimization algorithm (COA) simulates

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
EFFECT OF VARYING SWARM SIZE ON ACPA ALGORITHM PERFORMANCE

Swarm Size	mIoU(%)	mAcc(%)	mPrec(%)	mRec(%)	mF-Score(%)
30	84.13	90.78	91.16	90.78	90.77
60	**86.37**	**92.53**	**92.71**	**92.53**	**92.53**
90	84.88	91.18	91.58	91.18	91.17
120	85.27	91.06	91.86	91.06	91.03

TABLE IX
EFFECT OF VARYING INERTIA WEIGHT ON ACPA ALGORITHM PERFORMANCE

Inertia Weight	mIoU(%)	mAcc(%)	mPrec(%)	mRec(%)	mF-Score(%)
0.4	83.96	90.36	91.1	90.36	90.33
0.7	84.51	91.18	91.85	91.18	91.15
1	**86.37**	**92.53**	**92.71**	**92.53**	**92.53**
1.3	84.18	90.69	91.4	90.69	90.66

the hunting strategy of the coatis and effectively avoids falling into local optimization through global search [42]. Although COA has strong adaptability and robustness in complex search space, it has slow convergence speed and high computational cost, and is sensitive to parameter settings in large-scale data, which increases the difficulty of adjustment.

In contrast, the PSO algorithm achieves rapid convergence to global optimal solutions through individual and collective collaboration [43]. PSO features a simple structure, is straightforward to implement, and is applicable to various simple optimization tasks. However, PSO employs a relatively fixed search strategy: each particle can only adjust its trajectory based on its own best-found position and the swarm's global best position, lacking diversified exploration mechanisms. This singular strategy renders the algorithm susceptible to local optima entrapment, consequently limiting mIoU performance improvements. Therefore, the ACPA algorithm integrates the global search capabilities of the COA to compensate for PSO's deficiencies in exploration diversity while preserving PSO's rapid local convergence advantages.

In experiments conducted under the same conditions, the training time using StepLR was 26 710 s, while the training time using the ACPA algorithm was reduced to 22 917 s, resulting in a 14.2% improvement in model training speed. ACPA achieves efficient global search and rapid local convergence, leading to faster and better optimization results.

Furthermore, to investigate the stability of the ACPA algorithm's performance, we analyzed two key hyperparameters: population size and inertia weight. The experimental results demonstrated that, as shown in Table VIII, optimal performance (mIoU of 86.37%) was achieved with a population size of 60; populations that were too small failed to adequately cover the search space, adversely affecting the coati predation behavior simulation, while excessively large populations introduced computational redundancy and reduced efficiency. Regarding inertia weight, as shown in Table IX, a value of 1.0 yielded optimal performance; smaller weights diminished the influence of particles' historical velocity, limiting exploration capability, while larger weights caused excessive particle movement, hindering stabilization near optimal solutions. These findings validate the rationality of our parameter selection strategy.

TABLE X
RESULTS OF ABLATION EXPERIMENTS

Group	Method	mIoU (%)	mAcc (%)	mPrec (%)	mRec (%)	mF-Score (%)	Param (M)
A	PointMLP	60.17	74.29	81.27	74.29	73.84	**12.6**
B	+CSCA	64.11	78.24	80.74	78.24	78.12	16.7
C	+MSFA	74.1	84.11	85.55	84.11	84.03	12.6
D	+ACPA	82.1	88.88	89.85	88.88	88.86	12.6
E	+CSCA+MSFA	83.1	89.94	90.76	89.94	89.93	16.7
F	+CSCA+ACPA	82.52	89.39	90.29	89.39	89.39	16.7
G	+MSFA+ACPA	81.24	88.26	89.92	88.26	88.19	12.6
H	**RTreeNet**	**86.37**	**92.53**	**92.71**	**92.53**	**92.53**	16.7

D. Ablation Experiments

To validate the effectiveness of the proposed method, we conducted multiple ablation experiments on the RTreeNet architecture, with the results presented in Table X. First, we individually incorporated CSCA, MSFA, and ACPA into the model for validation. Next, we tested the combined effects of CSCA and MSFA, CSCA and ACPA, as well as MSFA and ACPA. Finally, these results were compared with those of the complete RTreeNet. To reduce the randomness of the data metrics, fivefold cross-validation was employed for training and testing the model.

To comprehensively showcase the network's performance under different module combinations, we selected five frontal views and three top–down views for visualization. Fig. 5(a) displays the original point cloud data of the rubber trees, highlighting the complex structure of the rubber trees and the background, along with significant noise. Fig. 5(b) presents the manually segmented results, where red indicates the rubber trees, blue represents the ground noise or adjacent leaves, and green denotes the misclassified points; a higher number of green points indicates poorer segmentation performance.

Fig. 5(c) shows the segmentation results of the baseline network PointMLP. Although this network demonstrates some segmentation capability, the results are suboptimal, with green points primarily concentrated in the crown and ground areas, reflecting its limitations in handling crown overlaps and ground noise. After incorporating the CSCA attention mechanism [Fig. 5(d)], the mIoU improved by 4%, and the model showed significant enhancement in removing ground noise, allowing for more precise separation of the ground and rubber trees. Further adding the ACPA optimization algorithm [Fig. 5(e)] facilitated faster convergence through global search and local optimization collaboration, leading to a noticeable increase in segmentation accuracy. Finally, after integrating the MSFA module in Fig. 5(f), the model excelled in capturing geometric features and addressing crown overlap issues, significantly reducing missegmentation and achieving optimal segmentation results.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

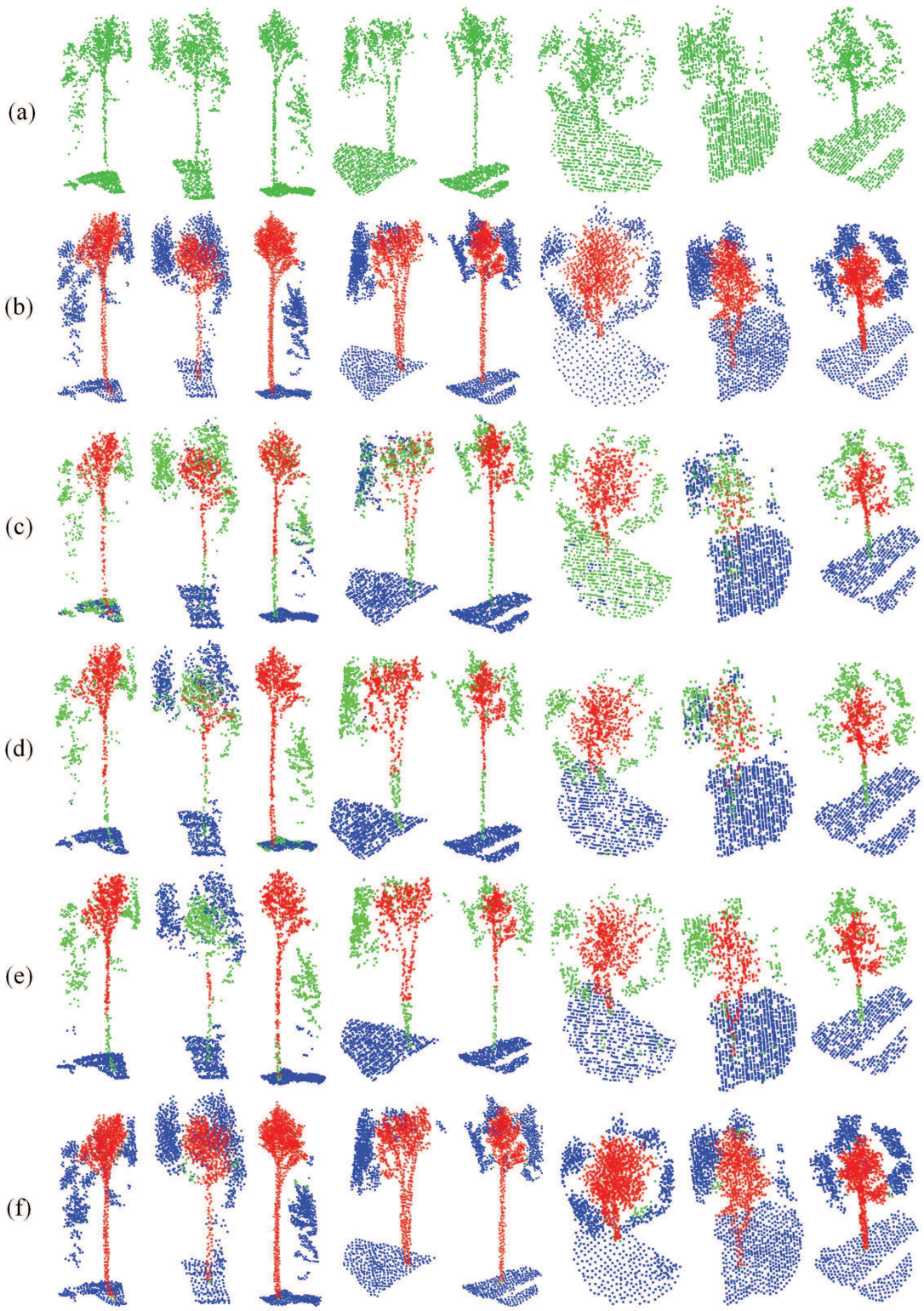


Fig. 5. Visualization of ablation experiments. (a) Original unsegmented rubber tree point cloud. (b) Ground-truth segmentation. (c) Segmentation results of the baseline network PointMLP. (d) Segmentation results after adding CSCA module. (e) Segmentation results after integrating CSCA attention mechanism and ACPA algorithm. (f) RTreeNet segmentation results. In segmentation results (c)–(f), red points represent the correctly identified rubber trees, blue points represent the correctly identified background (ground noise or adjacent foliage), and green points represent the misclassified points. More green points indicate poorer segmentation performance.

E. Compare With SOTA Methods

To verify the superiority of RTreeNet, we compared it with seven advanced deep learning point cloud segmentation networks on our self-built rubber tree dataset. These networks include PointNN [39], PointMLP [38], DRNet [44], GDANet [45], SPoTr [46], PointNext [47], and RISurConv [48], as well

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
CONTRAST EXPERIMENTAL RESULTS

Method	mIoU (%)	mAcc (%)	mPrec (%)	mRec (%)	mF-Score (%)	Param (M)
WSA(1979)	62.78	79.29	87.64	79.29	79.58	-
RGA(1994)	38.38	63.62	66.42	63.62	58.34	-
ECA(1967)	46.87	66.05	82.72	66.05	66.39	-
Treeiso(2022)	34.00	34.00	41.00	56.00	48.00	-
PointNN(2023)	31.96	56.54	31.96	56.54	40.84	3.9
PointMLP(2022)	60.17	74.29	81.27	74.29	73.84	12.6
DRNet(2020)	63.21	76.51	81.98	76.51	76.17	7.337
GDANet(2020)	65.50	78.71	83.66	78.71	78.70	**0.5826**
SpoTr(2023)	79.94	88.43	88.65	88.43	88.07	25.715
PointNext(2022)	82.36	90.41	90.38	90.41	90.28	0.9775
RISurConv(2024)	85.22	92.23	92.27	92.20	92.23	4.0556
Ours	**86.37**	**92.53**	**92.71**	**92.53**	**92.53**	16.777

TABLE XII
EXPERIMENTAL RESULTS ON THE SHAPEPARTNET DATASET

Shape/Model(%)	PointNet	PointNet++	PointMLP	DRNet	GDANet	PointNext	SpoTr	RTreeNet
Inst.mIoU	83.7	85.1	84.9	85.3	83.7	84.5	85.2	**85.6**
Cls.mIoU	80.4	81.9	82.5	82.3	80.3	79.8	82.0	**83.9**
aero	83.4	82.4	82.6	83.3	81.4	82.7	83.2	83.3
bag	78.7	79.0	78.3	86.0	77.9	70.2	79.4	83.3
cap	82.5	87.7	85.5	86.3	84.7	85.9	88.2	88.1
car	74.9	77.3	77.8	77.6	75.3	76.5	77.2	**79.6**
chair	89.6	90.8	89.7	90.2	88.9	90.4	90.6	90.4
earphones	73.0	71.8	77.5	75.5	72.4	69.5	72.6	**78.3**
guitar	91.5	91.0	91.2	91.4	90.7	88.9	90.5	**91.9**
knife	85.9	85.9	87.0	88.4	87.7	86.8	87.7	87.3
lamp	80.8	83.7	80.6	84.4	80.2	83.1	84.4	82.2
laptop	95.3	95.3	95.3	95.3	95.2	95.4	96.0	95.8
motorbike	65.2	71.6	72.0	68.2	58.3	58.8	67.2	**76.0**
mug	93.0	94.1	93.4	93.3	93.2	95.0	93.3	94.0
pistol	81.2	81.3	82.2	81.6	81.0	79.2	79.7	**84.8**
rocket	57.9	58.7	62.3	57.0	55.7	57.4	64.8	60.8
skateboard	72.8	76.4	81.1	74.7	79.3	74.4	73.8	**82.3**
table	80.6	82.6	83.5	83.1	82.4	82.8	83.0	83.3

as traditional point cloud segmentation algorithm watershed algorithm (WSA) [49], RGA [50], Euclidean clustering algorithm (ECA) [51] and Treeiso [52]. The experiment adopted the fivefold cross-validation method, and the results are shown in Table XI.

The experimental results show that RTreeNet outperforms other networks in all evaluation indicators, especially in mIoU, precision, recall, and F-score. Among them, RTreeNet's mIoU reached 86.3%, demonstrating its powerful ability to accurately segment rubber trees under complex backgrounds. In addition, RTreeNet's F-score is 92.5%, which further highlights its superior segmentation performance while comprehensively evaluating precision and recall rate, far exceeding other comparison networks.

F. Generalization Experiment

Generalization ability is an important indicator for evaluating model performance, reflecting the performance of the model on different datasets and its robustness and adaptability in segmentation tasks. To further verify the effectiveness of RTreeNet, we conducted experiments on three public datasets and compared the results with other advanced point cloud segmentation networks.

1) Generalization Experiments on the ShapePartNet Dataset: Table XII shows that RTreeNet exhibits exceptional generalization capability, achieving a classification average IoU (cls.mIoU) of 83.9% and an instance average IoU (ins.mIoU) of 85.6%, outperforming the other seven advanced point cloud segmentation networks. These improvements indicate that RTreeNet can more effectively capture and generalize the inherent complex geometric features within the ShapeNetPart dataset [53]. This evaluation confirms RTreeNet's superiority and reliability in handling diverse point cloud segmentation tasks, making it a robust choice for a wide range of application scenarios.

2) Generalization Experiments on the FOR-Instance Dataset: The publicly available FOR-instance benchmark dataset [54] was used as the source of ULS data. This dataset encompasses a wide range of forest types across five distinct geographical regions, including boreal forests (accounting for 42% of the total number of trees), temperate coniferous forests (5%), temperate mixed deciduous forests (29%), coniferous plantations (13%), and dry sclerophyllous forests (11%). The dataset comprises a total of 796 trees, and the detailed parameter characteristics [55] are presented in Table XIII.

To prepare single tree test data suitable for network evaluation, we used the Treeiso [52] unsupervised segmentation algorithm for preliminary processing. This process extracts point sets with the same identifier based on the scalar field value in the point cloud, generates independent single tree point cloud files, and obtains 100 suitable coarse segmentation datasets after careful screening, covering a variety of forest types in five different geographical locations, as shown in Fig. 6. In addition, in order to evaluate the generalization performance of different networks, we used a similar method in the previous data preprocessing to manually annotate and verify by experts to ensure high accuracy of the annotation, as shown in the second column of GT data in Fig. 7. This processing method is also applied to the NIBIO MLS dataset to ensure consistency in experimental evaluation.

The first five rows of Fig. 7 show the comparison results of RTreeNet and other advanced methods at five sites in the FOR-instance dataset. Blue and red points indicate correct classifications, and green points indicate misclassifications. RTreeNet shows good segmentation performance in various forest structures. Compared with the large number of misclassified points generated by traditional methods at the canopy boundaries, RTreeNet can accurately distinguish adjacent canopies in most cases.

The quantitative evaluation results are shown in Table XIV. RTreeNet achieves 68.42% mIoU and 86.98% mAcc on the FOR-instance dataset, which are 0.84% and 0.41% higher than the suboptimal methods GDANet and PointMLP, respectively. In addition, in the tests of CULS sparse forests and NIBIO dense forests, RTreeNet maintains high-precision segmentation, verifying its robustness to different tree densities and forest environments.

3) Generalization Experiments on the NIBIO MLS Dataset: To verify the cross-platform adaptability of RTreeNet, we used the NIBIO MLS dataset [56] for the third set of generalization experiments. This dataset was acquired using the GeoSLAM

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
SUMMARY OF THE MAIN FEATURES OF THE TWO GENERALIZATION DATASETS USED IN THIS STUDY

Dataset	Country	Forest type and main tree species	n trees and tree density	Mean tree height (m)	Data Type (sensor)	Average point density (pts m^{-2})
NIBIO MLS test data	Norway	Boreal coniferous forest (*Picea abies*, *Pinus sylvestris*, *Betula pendula*)	258; 1467 trees ha^{-1}	14.7	MLS (GeoSLAM ZEB-HORIZON)	20,000
FOR-instance test data	Norway, Czech Republic, Austria, New Zealand, Australia	Boreal coniferous forest (*Picea abies*), temperate coniferous forests (*Pinus sylvestris*), temperate mixed deciduous forest (*Fagus sylvatica*, *Acer platanoides*), plantation forest (*Pinus radiata*), dry sclerophyll forest (*Eucalyptus* sp.)	334; 425 trees ha^{-1}	21.6	ULS (Riegl VUX1 UAV and miniVUX)	6747

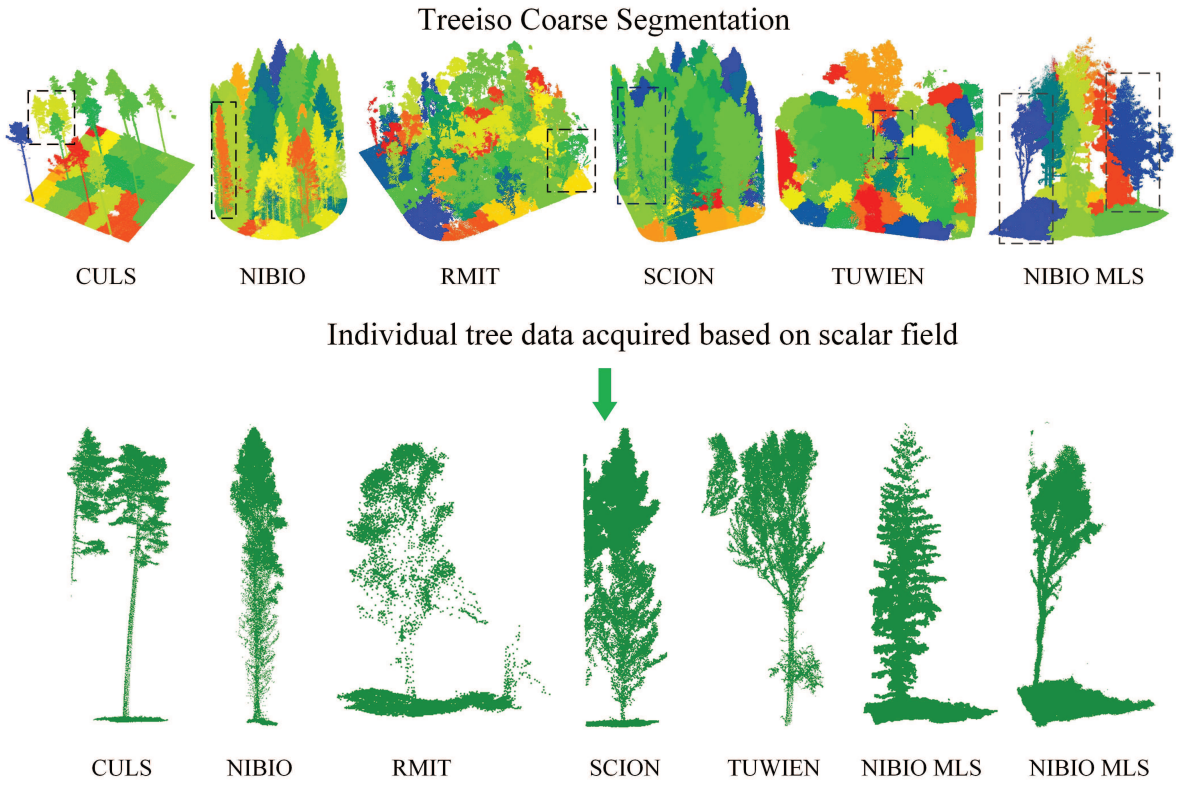


Fig. 6. (Top) Point cloud segmentation results of the Treeiso algorithm on two types of forest data: five sites in the FOR-instance dataset (CULS, NIBIO, RMIT, SCION, and TUWIEN) and NIBIO MLS data, where different colors represent different trees. (Bottom) Single tree point cloud for network evaluation based on scalar domain extraction.

ZEB-HORIZON device and represents the laser scanning characteristics of the ground perspective, which complements the drone perspective of the FOR-instance dataset. The data were collected from the boreal coniferous forest in 16 circular plots and contains 649 trees. The detailed characteristics are shown in Table XIII.

The last two rows of Fig. 7 show the segmentation results of RTreeNet on the NIBIO MLS data. The quantitative evaluation results are shown in Table XV. RTreeNet achieved 79.98% mIoU and 89.06% mAcc on this dataset, which are 1.40% and 1.33% higher than the second-best method GDANet, respectively. Compared with other methods, RTreeNet shows obvious advantages in the trunk-crown connection area and dense forest stands, demonstrating its robustness on different data collection platforms.

G. Structural Parameter Extraction

We selected rubber trees in two areas with different environmental features and measured and analyzed the structural

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

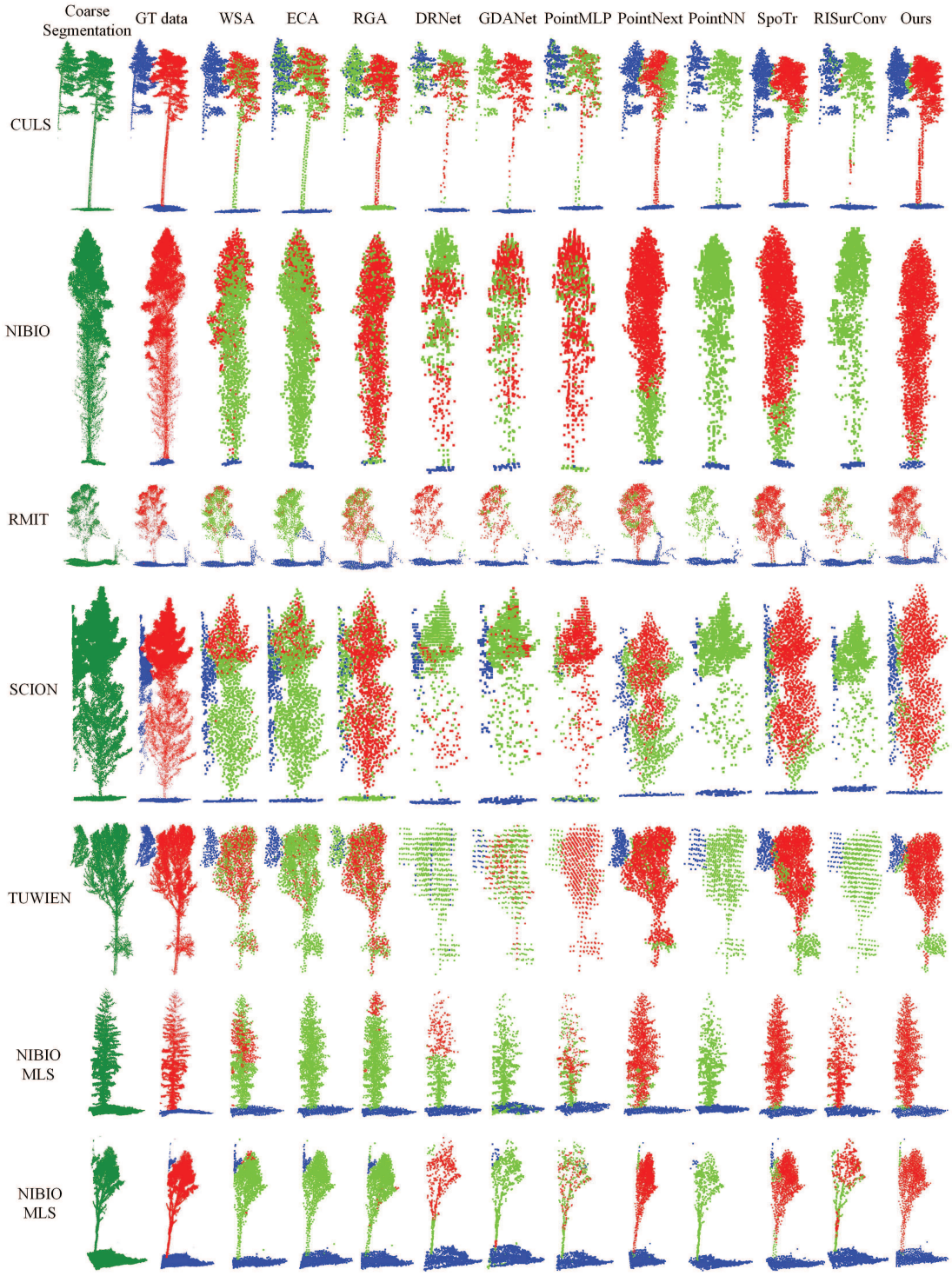


Fig. 7. Comparative visual analysis of various tree segmentation methodologies applied to the FOR-instance and NIBIO MLS datasets. The first five rows display test samples from five national sites within the FOR-instance dataset, while the final two rows showcase NIBIO MLS mobile laser scanning data. Correctly classified tree point clouds are represented in blue and red, whereas misclassifications appear in green. Moving from left to right, each column corresponds to a different algorithm or point cloud segmentation network. The rightmost column depicts the method proposed in this article, which demonstrates superior segmentation performance across most test scenarios.

parameters of the rubber trees based on the segmentation results from the network. Area 1 is located in a relatively flat region, with approximately 150 rubber trees distributed evenly. Area 2 is situated on a slope, containing about 200 rubber trees with a higher density and irregular distribution. By comparing the predicted values with the reference values, the generated scatterplots are shown in Fig. 8.

The structural characteristics of the rubber trees (tree height, DBH, and crown diameter) were extracted through point cloud

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

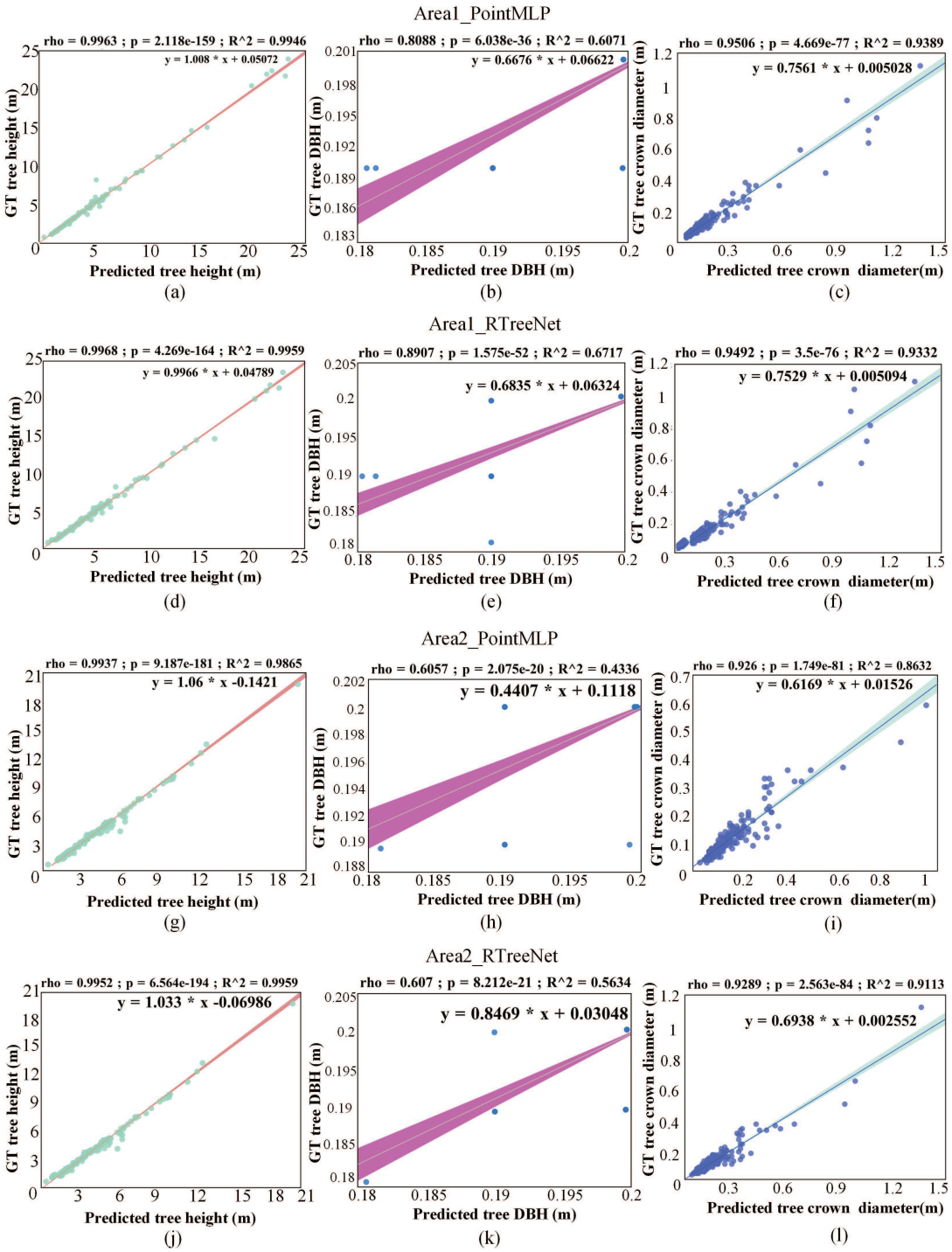


Fig. 8. (a)–(f) Scatterplots show the segmentation results of different networks for the predicted values of individual tree height, crown diameter, and DBH compared to the reference values in the first area. (g)–(l) Scatterplots for the second area.

feature extraction, which included data reading, ground model generation, tree height calculation, DBH fitting, and crown diameter estimation. The ground model was generated as a digital terrain model (DTM) by gridding the ground points. Tree height was calculated as the difference between the point cloud height and the DTM, while DBH and crown diameter were estimated through circular fitting and horizontal distribution range estimation [57].

As shown in Fig. 8(a), (d), (g), and (j), the predicted tree heights in both areas exhibited a very high correlation with the reference tree heights ($R^2 > 98\%$), indicating the accuracy and robustness of the model in tree height extraction. For the prediction of DBH, Fig. 8(b), (e), (h), and (k) demonstrates that the prediction accuracy is limited by the sparsity and noise of the point clouds, especially in cases where there are insufficient points for the DBH or significant noise, resulting in poor

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

TABLE XIV
COMPARISON OF GENERALIZATION PERFORMANCE OF DIFFERENT SEGMENTATION METHODS ON THE FOR-INSTANCE DATASET

Method	mIoU(%)	mAcc(%)	mPrec(%)	mRec(%)	mF-Score(%)
ECA	17.36	30.17	70.54	30.17	23.49
RGA	41.38	67.42	70	67.42	67.01
WSA	43.25	61.17	85.85	61.17	62.85
PointNN	33.77	33.78	72.12	33.78	17.08
DRNet	66.09	83.04	82.87	83.04	82.93
PointMLP	66.22	86.57	87.35	86.04	85.13
PointNext	66.42	83.12	82.53	83.12	82.67
SpoTr	67.38	84.03	83.65	84.03	83.77
GDANet	67.58	86.14	85.96	85.69	85.11
RISurConv	64.45	82.71	81.53	79.68	79.98
Ours	**68.42**	**86.98**	**87.19**	**86.51**	**85.85**

TABLE XV
COMPARISON OF GENERALIZATION PERFORMANCE OF DIFFERENT SEGMENTATION METHODS ON THE NIBIO MLS DATASET

Method	mIoU(%)	mAcc(%)	mPrec(%)	mRec(%)	mF-Score(%)
ECA	17.16	34.33	13.12	34.33	18.66
RGA	19.84	37.6	64.87	37.6	25.54
WSA	26.45	45.9	79.55	45.9	37.77
PointNN	48.46	48.46	23.49	48.46	31.64
PointNext	57.94	75.15	88.02	75.15	77.09
SpoTr	72.31	84.23	85.73	84.23	84
DRNet	54.53	71.75	79.44	71.75	70.02
PointMLP	76.2	85.33	85.34	85.33	85.32
GDANet	78.58	87.73	88.67	87.73	87.67
RISurConv	72.28	83.5	70.02	83.43	83.52
Ours	**79.98**	**89.06**	**89.54**	**89.06**	**89.05**

circular fitting outcomes. In some cases, there were not enough trunk points for fitting, leading to larger prediction errors for DBH. Nevertheless, the predicted values from RTreeNet were still closer to the reference values than those from PointMLP, showcasing its advantages in sparse data conditions.

For crown diameter prediction, as illustrated in Fig. 8(c), (f), (i), and (l), the accuracy of the predictions depended on the quality of single-tree segmentation and tree component classification. The relatively low segmentation accuracy of PointMLP resulted in errors in crown diameter estimates, particularly in Area 2, where the sparsity of the point cloud and increased tree density significantly affected segmentation accuracy. In contrast, the correlation coefficient (rho) for crown diameter estimation from RTreeNet was significantly higher than that from PointMLP, further validating RTreeNet's segmentation and classification capabilities.

In summary, RTreeNet outperformed PointMLP in complex environments, particularly demonstrating higher accuracy in predicting tree height and crown diameter. However, the accuracy of DBH predictions is still affected by the sparsity and noise of point cloud data, an issue that could be addressed in the future through further optimization of data collection or the introduction of new prediction methods.

H. Practical Application Analysis

To comprehensively evaluate the practical utility of our model, we conducted analyses from both computational cost and segmentation performance perspectives.

TABLE XVI
COMPUTATIONAL COST COMPARISON BETWEEN RTREENET AND POINTMLP (BATCH SIZE = 1)

Metrics	PointMLP	RTreeNet
Average inference time (ms/batch)	36.44	38.32
FLOPs (GFLOPs)	12.58	3.57
MACs (GMACs)	6.29	1.78
GPU peak memory (MB)	113.88	452.14
CPU memory usage (MB)	276.78	295.49
Computational efficiency (GFLOPs/s)	345.32	93.04

First, Table XVI presents a comparison of computational costs between RTreeNet and PointMLP. Under test conditions with a batch size of 1 and an average input data size of 4 MB, RTreeNet's average inference time is slightly longer than PointMLP's, yet still meets the requirements for real-time processing. Notably, RTreeNet exhibits significantly lower computational complexity (approximately 70% reduction) and correspondingly reduced MACs, indicating superior algorithmic efficiency. Although RTreeNet's peak GPU memory usage exceeds that of PointMLP, the CPU memory consumption remains comparable between the two methods. Considering RTreeNet's substantial improvement in segmentation accuracy, this tradeoff between computational resources and performance is reasonable and acceptable for practical applications.

To verify its actual segmentation effectiveness in real-world scenarios, we used a DJIM300RTK drone equipped with CBI-Lite to collect new, manually unlabeled rubber tree point cloud data in April 2024, as shown in Fig. 9(c). In Fig. 9(d), PointMLP exhibits significant errors when segmenting tree crowns, with blue circles indicating areas where background vegetation was incorrectly classified as rubber trees. Additionally, within densely distributed trees, PointMLP fails to accurately distinguish between tree trunks and crowns, resulting in poor segmentation quality. In Fig. 9(e), RTreeNet demonstrates improved overall segmentation performance, particularly with superior precision in segmenting tree crowns and trunks, with results displayed on the monitor, as shown in Fig. 9(b). However, the areas marked with blue circles also reveal RTreeNet's limitations, such as misclassifications that persist in boundary regions between trunks and crowns, and some point clouds being incorrectly categorized as part of the target tree due to dense overlapping. Comprehensive analysis indicates that RTreeNet achieves higher segmentation accuracy while maintaining reasonable computational overhead, with its favorable performance-cost balance making it more suitable for practical rubber tree point cloud segmentation applications.

I. Discussion and Shortcomings

The RTreeNet proposed in this study demonstrates significant advantages in the fine segmentation of rubber tree point clouds. As evidenced by the experimental results, RTreeNet surpasses existing methods across all evaluation metrics.

Despite these achievements, the RTreeNet model exhibits considerable scope for enhancement. From a model

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

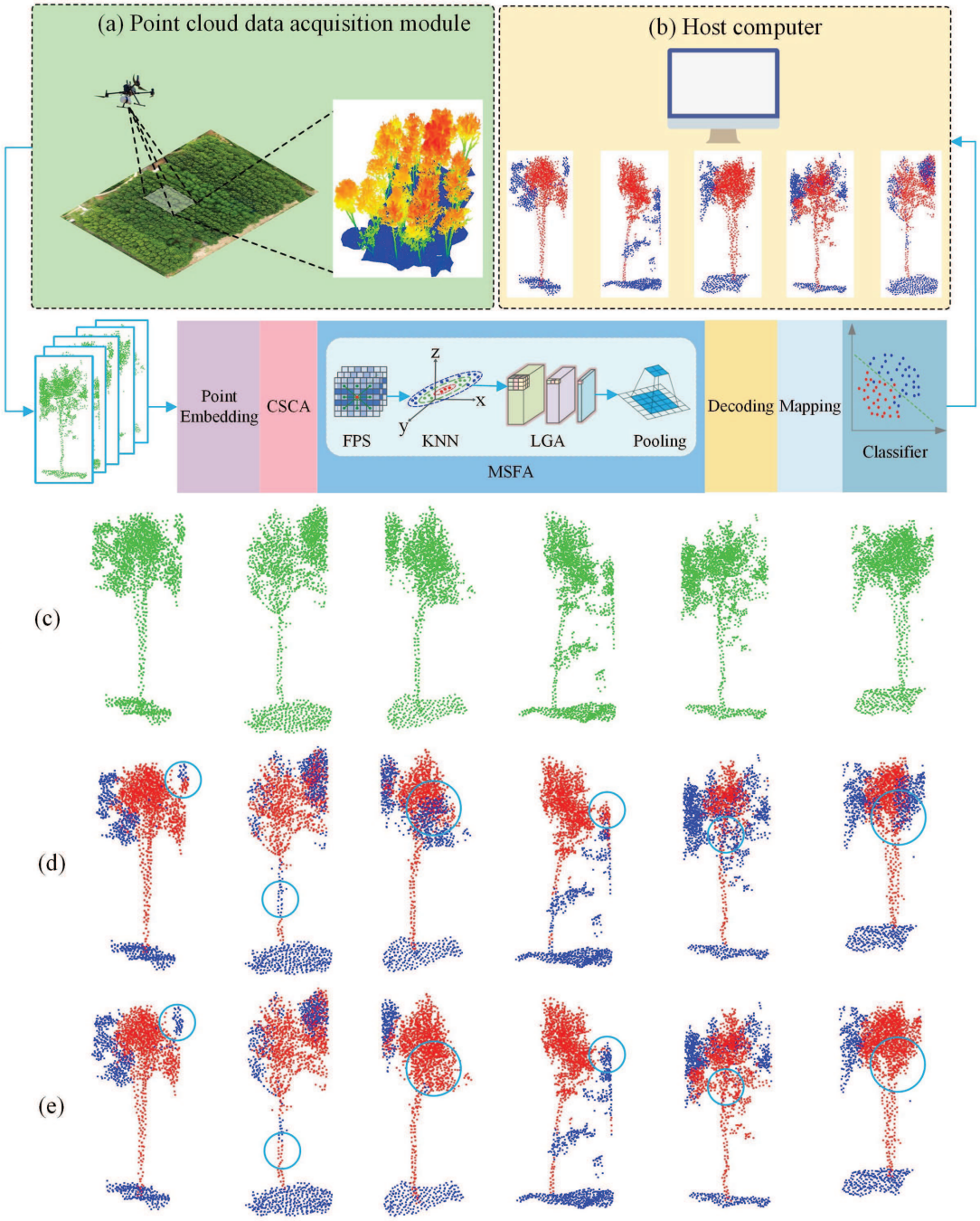


Fig. 9. (a) and (b) Practical application workflows, (c) unlabeled rubber tree point cloud data, (d) segmentation results of PointMLP, and (e) segmentation results of RTreeNet. In the segmentation results, red points represent regions predicted by the network as rubber trees, and blue points represent regions predicted by the network as background (ground interference or adjacent foliage).

complexity perspective, although the MSFA, CSCA, and ACPA modules enhance segmentation accuracy, they also substantially increase computational overhead, with the parameter count reaching 16.7M. Regarding generalization capability, the experimental results in Fig. 7 reveal that when adjacent tree crowns exhibit severe overlap (as observed in the SCION site data), RTreeNet struggles to precisely delineate individual tree boundaries. As shown in Fig. 10, in extremely dense plantation environments where multiple rubber trees highly overlap (Fig. 10(a)), although the model can identify the main tree structures, it still faces challenges in boundary regions with severe crown overlap.

We found that the performance of RTreeNet is largely constrained by the quality and availability of training data, rather than simply by the limitations of the model architecture. Although our network performs well on self-built and public datasets, high-quality labeled training data are still scarce and difficult to obtain for extremely complex forest environments, especially in areas with severe crown overlap, large terrain undulations, and high species diversity.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

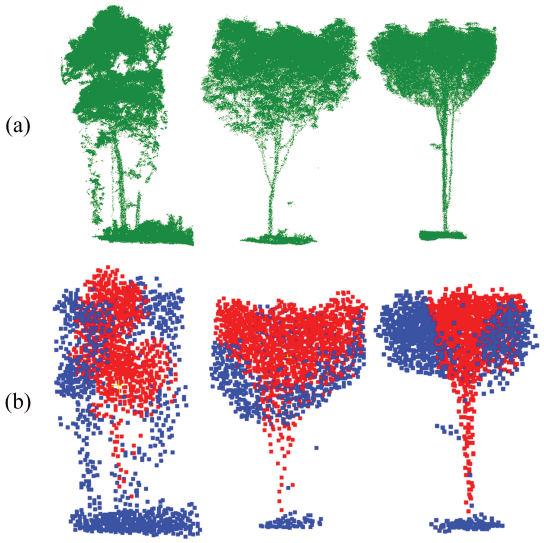


Fig. 10. Segmentation challenges in high-density planting environments. (a) Point cloud of three closely overlapping rubber trees. (b) Segmentation results, where red points represent regions predicted by the network as rubber trees, and blue points represent regions predicted by the network as background (ground interference or adjacent foliage).

Future research directions should focus on simplifying the model structure to reduce computational costs while enhancing the ability to handle complex geometric structures. In addition, developing better data augmentation strategies and heterogeneous feature fusion mechanisms to fully utilize the complementarity of multisource information is expected to further improve the segmentation accuracy in dense forest environments.

V. Conclusion

This article presents a novel point cloud segmentation network, RTreeNet, designed for the fine segmentation and structural parameter extraction of individual rubber trees. The specific conclusions are as follows.

1) The three core modules proposed have their own unique functions: the CSCA module effectively improves the ground noise separation ability by calculating the feature cosine similarity in the vertical and horizontal directions; the ACPA algorithm optimizes the learning rate adjustment mechanism and significantly accelerates the model convergence speed; and the MSFA module specifically solves the segmentation accuracy problem in the overlapping crown area.
2) Through systematic experimental evaluation, RTreeNet shows superior performance in accurately segmenting a single point cloud rubber tree, which not only provides a reliable tool for accurate measurement of forestry parameters, but also provides technical support for practical applications such as rubber forest resource assessment, growth monitoring, and refined management.

Nevertheless, the model faces challenges in computational complexity and adaptability to specific scenarios. Future research will focus on optimizing the model architecture to reduce the computational burden, and explore cross-modal fusion methods for multisource heterogeneous data, integrating point cloud geometric features with semantic information of high-resolution optical images and hyperspectral data, and further promoting the practical application of point cloud segmentation technology in fine forestry management.

References

[1] T. Yun et al., "Rubber tree crown segmentation and property retrieval using ground-based mobile LiDAR after natural disturbances," *Remote Sens.*, vol. 11, no. 8, p. 903, Apr. 2019.
[2] S. Li and L. Ye, "Multi-level thresholding image segmentation for rubber tree secant using improved Otsu's method and snake optimizer," *Math. Biosci. Eng.*, vol. 20, no. 6, pp. 9645–9669, 2023.
[3] Z. Xie, J. Wu, W. Tang, and Y. Liu, "Advancing image segmentation with DBO-Otsu: Addressing rubber tree diseases through enhanced threshold techniques," *PLoS ONE*, vol. 19, no. 3, Mar. 2024, Art. no. e0297284.
[4] L. Fan, Y. Zhou, H. Liu, Y. Li, and D. Cao, "Combining Swin transformer with UNet for remote sensing image semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5530111.
[5] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, and D. Terzopoulos, "Image segmentation using deep learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3523–3542, Jul. 2022.
[6] T. Xiao, Y. Liu, Y. Huang, M. Li, and G. Yang, "Enhancing multiscale representations with transformer for remote sensing image semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5605116.
[7] J. Zhang, H. Liu, K. Yang, X. Hu, R. Liu, and R. Stiefelhagen, "CMX: Cross-modal fusion for RGB-X semantic segmentation with transformers," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 14679–14694, Dec. 2023.
[8] J. Wang et al., "Individual rubber tree segmentation based on ground-based LiDAR data and faster R-CNN of deep learning," *Forests*, vol. 10, no. 9, p. 793, Sep. 2019.
[9] Y. Chi, C. Wang, Z. Chen, and S. Xu, "TCSNet: A new individual tree crown segmentation network from unmanned aerial vehicle images," *Forests*, vol. 15, no. 10, p. 1814, Oct. 2024.
[10] H. Chen, D. Sun, W. Liu, H. Wu, M. Liang, and P. X. Liu, "A novel approach to the extraction of key points from 3-D rigid point cloud using 2-D images transformation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5703815.
[11] L. He et al., "GFOICP: Geometric feature optimized iterative closest point for 3-D point cloud registration," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5704217.
[12] J. Li, Q. Hu, and M. Ai, "Point cloud registration based on one-point RANSAC and scale-annealing biweight estimation," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 11, pp. 9716–9729, Nov. 2021.
[13] C. Wang, Y. Gu, and X. Li, "A robust multispectral point cloud generation method based on 3-D reconstruction from multispectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5407612.
[14] F. Morsdorf, E. Meier, B. Kötz, K. I. Itten, M. Dobbertin, and B. Allgöwer, "LiDAR-based geometric reconstruction of boreal type forest stands at single tree level for forest and wildland fire management," *Remote Sens. Environ.*, vol. 92, no. 3, pp. 353–362, Aug. 2004.
[15] H. Lee, K. C. Slatton, B. E. Roth, and W. P. Cropper, "Adaptive clustering of airborne LiDAR data to segment individual tree crowns in managed pine forests," *Int. J. Remote Sens.*, vol. 31, no. 1, pp. 117–139, Jan. 2010.
[16] Z. Hui, P. Cheng, B. Yang, and G. Zhou, "Multi-level self-adaptive individual tree detection for coniferous forest using airborne LiDAR," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 114, Nov. 2022, Art. no. 103028.
[17] W. Chen, X. Hu, W. Chen, Y. Hong, and M. Yang, "Airborne LiDAR remote sensing for individual tree forest inventory using trunk detection-aided mean shift clustering techniques," *Remote Sens.*, vol. 10, no. 7, p. 1078, Jul. 2018.
[18] R. Q. Charles, H. Su, M. Kaichun, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 77–85.
[19] C. R. Qi, Y. Li, H. Su, and L. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5105–5114.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

[20] Z. Ma, Y. Dong, J. Zi, F. Xu, and F. Chen, "Forest-PointNet: A deep learning model for vertical structure segmentation in complex forest scenes," *Remote Sens.*, vol. 15, no. 19, p. 4793, Sep. 2023.
[21] D.-H. Kim, C.-U. Ko, D.-G. Kim, J.-T. Kang, J.-M. Park, and H.-J. Cho, "Automated segmentation of individual tree structures using deep learning over LiDAR point cloud data," *Forests*, vol. 14, no. 6, p. 1159, Jun. 2023.
[22] L. Chang, H. Fan, N. Zhu, and Z. Dong, "A two-stage approach for individual tree segmentation from TLS point clouds," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 8682–8693, 2022.
[23] P. Wilkes et al., "TLS2trees: A scalable tree segmentation pipeline for TLS data," *Methods Ecol. Evol.*, vol. 14, no. 12, pp. 3083–3099, Dec. 2023.
[24] J. Henrich, J. van Delden, D. Seidel, T. Kneib, and A. S. Ecker, "TreeLearn: A deep learning method for segmenting individual trees from ground-based LiDAR forest point clouds," *Ecolog. Informat.*, vol. 84, Dec. 2024, Art. no. 102888.
[25] A. Harikumar, F. Bovolo, and L. Bruzzone, "A local projection-based approach to individual tree detection and 3-D crown delineation in multistoried coniferous forests using high-density airborne LiDAR data," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1168–1182, Feb. 2019.
[26] Y. Hao, F. R. A. Widagdo, X. Liu, Y. Liu, L. Dong, and F. Li, "A hierarchical region-merging algorithm for 3-D segmentation of individual trees using UAV-LiDAR point clouds," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5701416.
[27] J. Hu et al., "A robust deep learning approach for the quantitative characterization and clustering of peach tree crowns based on UAV images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4408613.
[28] Y. Duan, C. Yang, H. Chen, W. Yan, and H. Li, "Low-complexity point cloud denoising for LiDAR by PCA-based dimension reduction," *Opt. Commun.*, vol. 482, Mar. 2021, Art. no. 126567.
[29] N. Diaz, O. Gallo, J. Caceres, and H. Porras, "Real-time ground filtering algorithm of cloud points acquired using terrestrial laser scanner (TLS)," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 105, Dec. 2021, Art. no. 102629.
[30] H. Ren, M. Wang, W. Li, C. Liu, and M. Zhang, "Adaptive patchwork: Real-time ground segmentation for 3D point cloud with adaptive partitioning and spatial–temporal context," *IEEE Robot. Autom. Lett.*, vol. 8, no. 11, pp. 7162–7169, Nov. 2023.
[31] G. Ioannou, T. Tagaris, and A. Stafylopatis, "AdaLip: An adaptive learning rate method per layer for stochastic optimization," *Neural Process. Lett.*, vol. 55, no. 5, pp. 6311–6338, Oct. 2023.
[32] H. Kabiri, Y. Ghanou, H. Khalifi, and G. Casalino, "AMAdam: Adaptive modifier of Adam method," *Knowl. Inf. Syst.*, vol. 66, no. 6, pp. 3427–3458, Jun. 2024.
[33] A. C. Carrilho, M. Galo, and R. C. Santos, "Statistical outlier detection method for airborne LiDAR data," *Int. Arch. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 1, pp. 87–92, Sep. 2018.
[34] W. Zhang et al., "An easy-to-use airborne LiDAR data filtering method based on cloth simulation," *Remote Sens.*, vol. 8, no. 6, p. 501, Jun. 2016.
[35] L. Liu, S. Lim, X. Shen, and M. Yebra, "A hybrid method for segmenting individual trees from airborne LiDAR data," *Comput. Electron. Agricult.*, vol. 163, Aug. 2019, Art. no. 104871.
[36] X. Chen, K. Jiang, Y. Zhu, X. Wang, and T. Yun, "Individual tree crown segmentation directly from UAV-borne LiDAR data using the PointNet of deep learning," *Forests*, vol. 12, no. 2, p. 131, Jan. 2021.
[37] D. Girardeau-Montaut. (2022). *CloudCompare: 3D Point Cloud and Mesh Processing Software*. [Online]. Available: https://www.cloudcompare.org/
[38] X. Ma, C. Qin, H. You, H. Ran, and Y. Fu, "Rethinking network design and local geometry in point cloud: A simple residual MLP framework," 2022, *arXiv:2202.07123*.
[39] R. Zhang et al., "Parameter is not all you need: Starting from non-parametric networks for 3D point cloud analysis," 2023, *arXiv:2303.08134*.
[40] W. Cai et al., "Graph-structured convolution-guided continuous context threshold-aware networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5527618.
[41] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *Comput. Sci.*, vol. 2014, pp. 1–15, Dec. 2014.
[42] M. Dehghani, Z. Montazeri, E. Trojovská, and P. Trojovský, "Coati optimization algorithm: A new bio-inspired metaheuristic algorithm for solving optimization problems," *Knowl.-Based Syst.*, vol. 259, Jan. 2023, Art. no. 110011.
[43] Y.-J. Zheng, H.-F. Ling, and Q. Guan, "Adaptive parameters for a modified comprehensive learning particle swarm optimizer," *Math. Problems Eng.*, vol. 2012, no. 1, Jan. 2012, Art. no. 207318.
[44] S. Qiu, S. Anwar, and N. Barnes, "Dense-resolution network for point cloud classification and segmentation," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 3812–3821.
[45] M. Xu, J. Zhang, Z. Zhou, M. Xu, X. Qi, and Y. Qiao, "Learning geometry-disentangled representation for complementary understanding of 3D object point cloud," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 3056–3064.
[46] J. Park, S. Lee, S. Kim, Y. Xiong, and H. J. Kim, "Self-positioning point-based transformer for point cloud understanding," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 21814–21823.
[47] G. Qian et al., "PointNeXt: Revisiting PointNet++ with improved training and scaling strategies," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 23192–23204.
[48] Z. Zhang, L. Yang, and Z. Xiang, "RISurConv: Rotation invariant surface attention-augmented convolutions for 3D point cloud classification and segmentation," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 93–109.
[49] S. Beucher, "Use of watersheds in contour detection," in *Proc. Int. Workshop Image Process.*, 1979.
[50] R. Adams and L. Bischof, "Seeded region growing," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 16, no. 6, pp. 641–647, Jun. 1994.
[51] J. B. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. Berkeley Symp. Math. Statist. Probab.*, vol. 1, 1967, pp. 281–297.
[52] Z. Xi and C. Hopkinson, "3D graph-based individual-tree isolation (Treeiso) from terrestrial laser scanning point clouds," *Remote Sens.*, vol. 14, no. 23, p. 6116, 2022.
[53] A. X. Chang et al., "ShapeNet: An information-rich 3D model repository," 2015, *arXiv:1512.03012*.
[54] S. Puliti, "*FOR-instance: A UAV laser scanning benchmark dataset for semantic and instance segmentation of individual trees*," doi: 10.5281/zenodo.8287792.
[55] M. Wielgosz, S. Puliti, B. Xiang, K. Schindler, and R. Astrup, "SegmentAnyTree: A sensor and platform agnostic deep learning model for tree segmentation using laser scanning data," *Remote Sens. Environ.*, vol. 313, Nov. 2024, Art. no. 114367.
[56] M. Wielgosz, S. Puliti, P. Wilkes, and R. Astrup, "Point2Tree(P2T)—Framework for parameter tuning of semantic and instance segmentation used with mobile laser scanning data in coniferous forest," *Remote Sens.*, vol. 15, no. 15, p. 3737, Jul. 2023. [Online]. Available: https://www.mdpi.com/2072-4292/15/15/3737
[57] B. Xiang et al., "Automated forest inventory: Analysis of high-density airborne LiDAR point clouds with 3D deep learning," *Remote Sens. Environ.*, vol. 305, May 2024, Art. no. 114078.

Li Li is currently pursuing the master's degree with the Central South University of Forestry and Technology, Changsha, China.

His main research interests include LiDAR, point cloud segmentation, and deep learning.

Zilin Ye received the master's degree in forestry from the Central South University of Forestry and Technology, Changsha, China, in 2021, where he is currently pursuing the Ph.D. degree in forestry.

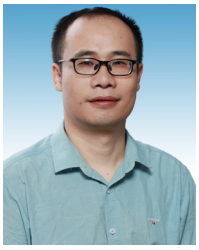
His research interests focus on forest management and forest remote sensing.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.

Hao Li is currently pursuing the master's degree with the Central South University of Forestry and Technology, Changsha, China.

His main research interests include machine learning, computer vision, and remote sensing applications.

Miying Yan is currently an Associate Professor with the Central South University of Forestry and Technology, Changsha, China. She is chairing some research projects. Her research interests include artificial intelligence and intelligent control.



Guoxiong Zhou received the Ph.D. degree in control science and engineering from Central South University, Changsha, China, in 2010.

He is currently a Professor with the Central South University of Forestry and Technology, Changsha. He is chairing some research projects. His research interests include image processing, artificial intelligence, and intelligent control.

Xiangjun Wang received the Ph.D. degree in crop genetics and breeding from Shihezi University, Shihezi, China, in 2011.

He is currently working with the Rubber Research Institute, Chinese Academy of Tropical Agricultural Sciences, Haikou, China. His research interests mainly focus on the genetics and breeding of rubber trees.

Hengrui Wang is currently pursuing the master's degree with the Central South University of Forestry and Technology, Changsha, China.

His main research interests include individual tree segmentation, semantic segmentation, and point cloud processing.

Mingjie Lv is currently pursuing the M.S. and Ph.D. degrees with the School of Automation, Central South University, Changsha, China.

His research interests include graph networks, industrial intelligence, pattern recognition, and machine learning.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 06:57:45 UTC from IEEE Xplore. Restrictions apply.